# RAJALAKSHMI ENGINEERING COLLEGE
RAJALAKSHMI NAGAR, THANDALAM – 602 105

RAJALAKSHMI ENGINEERING COLLEGE

CS23334 – Fundamentals of Data Science Lab

**Laboratory Record Note Book**

Name : SWETHA.S

Year / Branch / Section : II CSE

University Register No. : 2116250701772

College Roll No. : 250701772

Semester : III

Academic Year : 2026-27

# DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CS23334 – Fundamentals of Data Science Lab

| | |
|---|---|
| NAME | SWETHA.S |
| ROLL NO | 250701772 |
| YEAR | II |
| SEC | A |

**RAJALAKSHMI ENGINEERING COLLEGE**
**An Autonomous Institution**

## BONAFIDE CERTIFICATE

**Name:** SWETHA.S

**Academic Year:** 2026-27 **Semester:** III **Branch:** CSE

**Register No.** 250701772

*Certified that this is the bonafide record of work done by the above student in the* Fundamental of Data Science *Laboratory during the academic year 2026 – 2027.*

3/11/26

**Signature of Faculty in-charge**

**Submitted for the Practical Examination held on .....**

Internal Examiner

External Examiner

# INDEX

| Sl.NO | Experiment | Date | GitHub |
|---|---|---|---|
| 1.a | Analyze the trend of data science job postings over the last decade for the given data on the number of data science job postings each year. Use pandas for data manipulation and matplotlib /seaborn for visualization. | 29/07/26 | https://github.com/swetha-sukumar/fds-lab-workouts |
| 1.b | Analyze and visualize the distribution of various data science roles (Data Analyst, Data Engineer, Data Scientist, etc.) from a dataset. | 29/07/26 | |
| 1.c | Conduct an experiment to differentiate Structured , Un-structured and Semi structured data based on data sets given. | 05/08/26 | |
| 1.d | Conduct an experiment to encrypt and decrypt given sensitive data Use the cryptography library fernet. | 05/08/26 | |
| 2 | Upload and Analyze the data set given in csv format and perform data preprocessing and visualization. | 12/08/26 | |
| 3.a | Conduct an experiment to show data visualization using line plot | 19/08/26 | |
| 3.b | Conduct an experiment to show data visualization using bar chart | 19/08/26 | |
| 3.c | Conduct an experiment to show data visualization using pie chart | 19/08/26 | |
| 4 | To Count the frequency of occurrence of a word in a body of text is often needed during text processing. | 26/08/26 | |
| 5 | Data Collection and Initial Exploration | 16/09/26 | |
| 6 | Time Series Analysis of Website Traffic | 16/09/26 | |

| | | | |
|---|---|---|---|
| 7 | Random Sampling and Sampling Distribution | 23/09/26 | |
| 8 | Conduct Z test for the Data given using Python | 23/09/26 | |
| 9 | Experiment to understand Matplotlib library use cases in Data Science through visualization. | 23/09/26 | |
| 10 | Experiment to understand array function in Data science. | 23/09/26 | https://github.com/swetha-sukumar/fds-lab-workouts |
| 11 | Experiment to understand pandas library use cases in Data science. | 30/09/26 | |
| 12 | Experiment to detect outliers in a given data set. | 30/09/26 | |
| 13 | Experiment to handle missing values and inappropriate data in a given data set. | 30/09/26 | |
| 14 | Experiment to understand feature scaling | 30/09/26 | |
| 15 | Experiment to understand the data preprocessing in Data science | 30/09/26 | |
| 16 | Experiment to understand EDA-Quantitative and Qualitative analysis. | 07/10/26 | |
| 17 | Experiment to understand Linear Regression for a given data set. | 07/10/26 | |
| 18 | Experiment to understand KNN algorithm for a given data set. | 07/10/26 | https://github.com/swetha-sukumar/fds-lab-workouts |
| 19 | Experiment to understand Logistic Regression for a given data set. | 07/10/26 | https://github.com/swetha-sukumar/fds-lab-workouts |
| 20 | Experiment to understand K-means clustering algorithm for a given data set. | 07/10/26 | https://github.com/swetha-sukumar/fds-lab-workouts |

**EXPERIMENT NO:1**

**Exp No:1.a**

**Analyze the trend of data science job postings over the last Decade**

**AIM:**

To analyze the trend of data science job postings over the last Decade.

**Program code:**

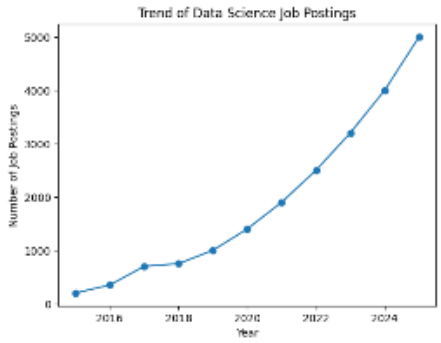
```
import pandas as pd
import matplotlib.pyplot as plt

data = {'Year': list(range(2015, 2026)),
        'Job Postings': [200, 350, 700, 750, 1000, 1400, 1900, 2500, 3200, 4000, 5000]}

df = pd.DataFrame(data)

plt.plot(df['Year'], df['Job Postings'], marker='o')
plt.title('Trend of Data Science Job Postings')
plt.xlabel('Year')
plt.ylabel('Number of Job Postings')
plt.show()
```

**Output:**




**Result:**

Thus, the trend of Data Science job postings from 2015 to 2025 was successfully analyzed and visualized using a line plot.

**Exp No:1.b**

**Analyze and visualize the distribution of various data science roles (Data Analyst, Data Engineer, Data Scientist, etc.) from a dataset**

**Aim:**

To Analyze and visualize the distribution of various data science roles (Data Analyst, Data Engineer, Data Scientist, etc.) from a dataset

**Program code:**

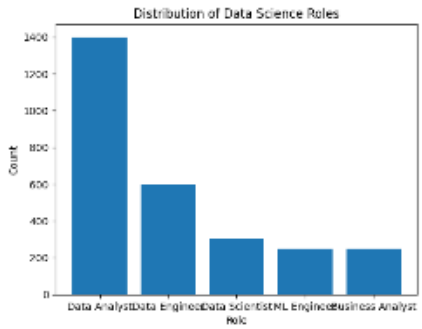
```
import matplotlib.pyplot as plt

roles = ['Data Analyst', 'Data Engineer', 'Data Scientist',
        'ML Engineer', 'Business Analyst']

counts = [1400, 600, 300, 250, 250]

plt.bar(roles, counts)
plt.title('Distribution of Data Science Roles')
plt.xlabel('Role')
plt.ylabel('Count')
plt.show()
```

**Output:**




**Result:**

Thus, the distribution of different Data Science roles was successfully analyzed and visualized using a bar chart.

## Exp No:1.c Conduct an experiment to differentiate Structured , Un-structured and Semi

## structured data based on data sets given.

**Aim:**

To conduct an experiment to differentiate Structured , Un-structured and Semi structured data based on data sets given.

**Program code:**

```
import pandas as pd
structured_data = pd.DataFrame({
    'ID': [101, 102, 103],
    'Name': ['David', 'Emma', 'John'],
    'Age': [28, 32, 27]
})
print("Structured Data:\n", structured_data)
unstructured_data = "This is a sample document containing text, images and video
content."
print("In Unstructured Data:\n", unstructured_data)
semi_structured_data = {
```

```
    'ID': 101,
    'Name': 'David',
    'Attributes': {
        'Height': 170,
        'Weight': 65
    }
}

print("In Semi-structured Data:\n", semi_structured_data)
```

**Output:**

```
Structured Data:
    ID   Name  Age
0  101  David   28
1  102   Emma   32
2  103   John   27
In Unstructured Data:
 This is a sample document containing text, images and video content.
In Semi-structured Data:
 {'ID': 101, 'Name': 'David', 'Attributes': {'Height': 170, 'Weight': 65}}
```

**Result:**

Thus, the given data was successfully classified into structured, unstructured, and semi-structured data.

**Exp No:1.d Conduct an experiment to encrypt and decrypt given sensitive data.**

**Aim:**

To conduct an experiment to encrypt and decrypt given sensitive data.

**Program code:**

```
from cryptography.fernet import Fernet
key = Fernet.generate_key()
cipher_suite = Fernet(key)
plain_text = b"Computer Science Engineering"
cipher_text = cipher_suite.encrypt(plain_text)
decrypted_text = cipher_suite.decrypt(cipher_text)
print("Original Data:", plain_text)
print("Encrypted Data:", cipher_text)
print("Decrypted Data:", decrypted_text)
```

**Output:**

```
Original Data: b'Computer Science Engineering'
Encrypted Data: b'gAAAAABqfAgZMLf7ngrhpWlRD7Yv95Q-OExrIRSxq8dx-
BpP36fB_8_X1JT8jynGL7CO4Q8xBTZMtke4q3lBzrAaXKvfBk11EUUiDgtMtkemVrZn
mLwZyT0='
Decrypted Data: b'Computer Science Engineering'
```

**Result:**

Thus, the given data was successfully encrypted and decrypted using the Fernet encryption technique.

**EXPERIMENT NO:2**

## Upload and Analyze the data set given in csv format and perform data preprocessing and visualization.

**Aim:**

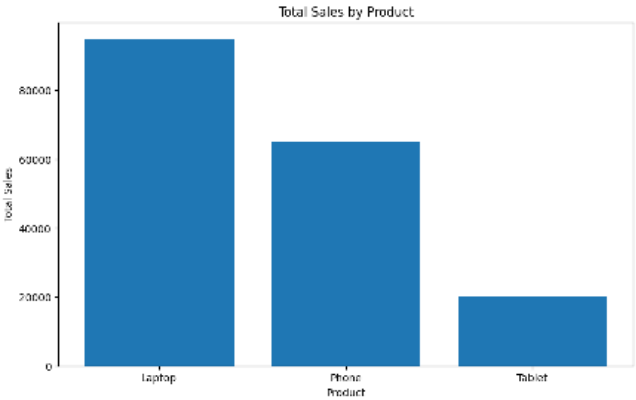
To Upload and Analyze the data set given in csv format and perform data preprocessing and visualization.

Program code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
file_path='C:\market_data.csv'
df = pd.read_csv(file_path)
print(df.head())
print(df.isnull().sum())
df['Sales'].fillna(df['Sales'].mean(), inplace=True)
df.dropna(subset=['Product', 'Quantity', 'Region'], inplace=True)
print(df.describe())
product_summary = df.groupby('Product').agg({
   'Sales':'sum',
   'Quantity':'sum'
}).reset_index()

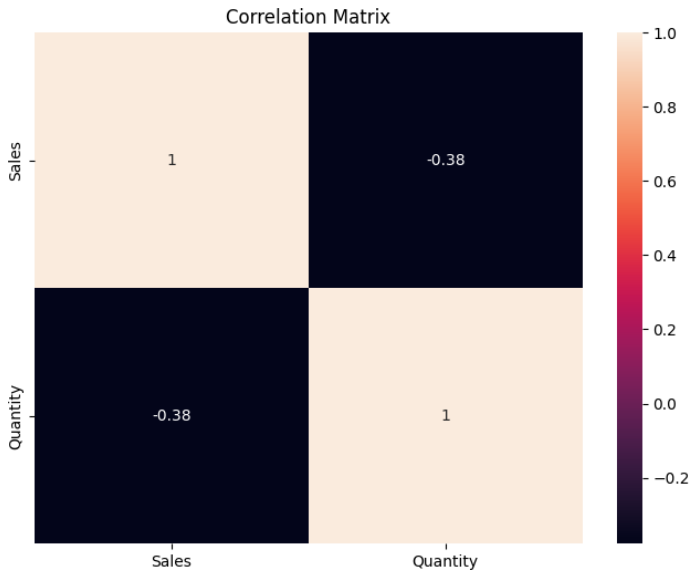
print(product_summary)
plt.figure(figsize=(10,6))
plt.bar(product_summary['Product'], product_summary['Sales'])
plt.xlabel('Product')
plt.ylabel('Total Sales')
plt.title('Total Sales by Product')
plt.show()
df['Date'] = pd.to_datetime(df['Date'])
sales_over_time = df.groupby('Date').agg({'Sales':'sum'}).reset_index()
plt.figure(figsize=(10,6))
plt.plot(sales_over_time['Date'], sales_over_time['Sales'])
plt.xlabel('Date')
plt.ylabel('Total Sales')
plt.title('Sales Over Time')
plt.show()
pivot_table = df.pivot_table(values='Sales', index='Region', columns='Product',
aggfunc=sum, fill_value=0)
print(pivot_table)
correlation_matrix = df.corr()
print(correlation_matrix)
import seaborn as sns
plt.figure(figsize=(8,6))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm')
plt.title('Correlation Matrix')
```

plt.show()
Output:










Result:
Thus, the given CSV dataset was successfully uploaded, preprocessed, analyzed, and visualized using Python.

EXPERIMENT NO:3
3.a)

# Conduct an experiment to show data visualization using line plot

**Description:**

Take any sample data either through csv file or data fetched directly through code.

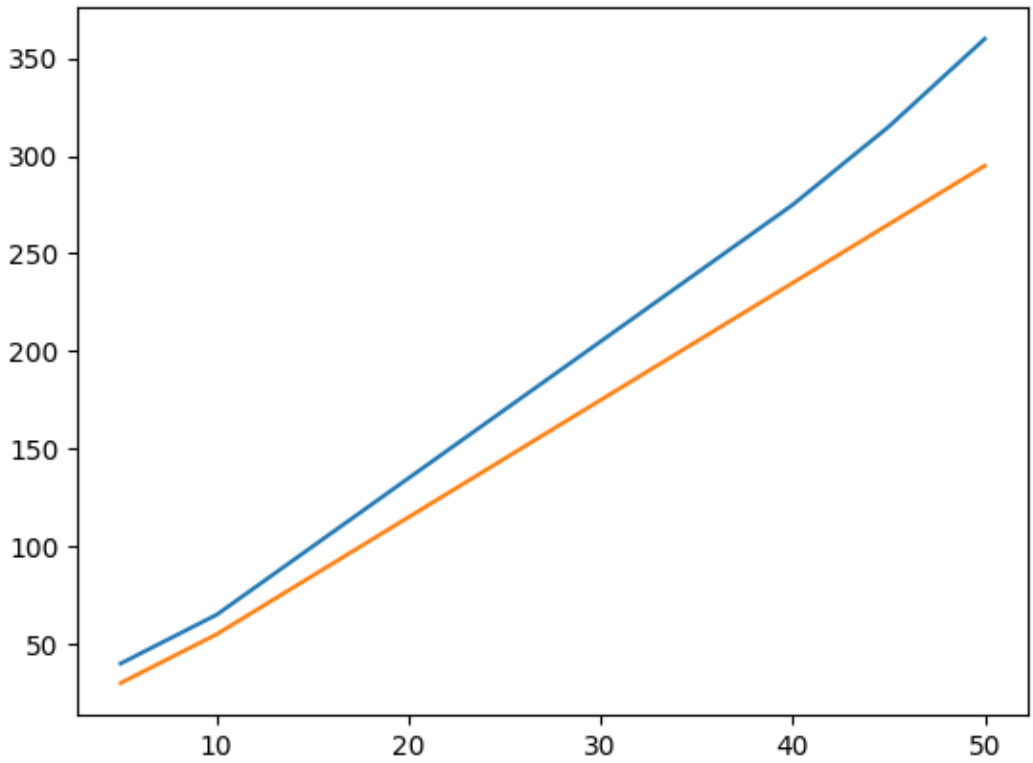
**Program:**

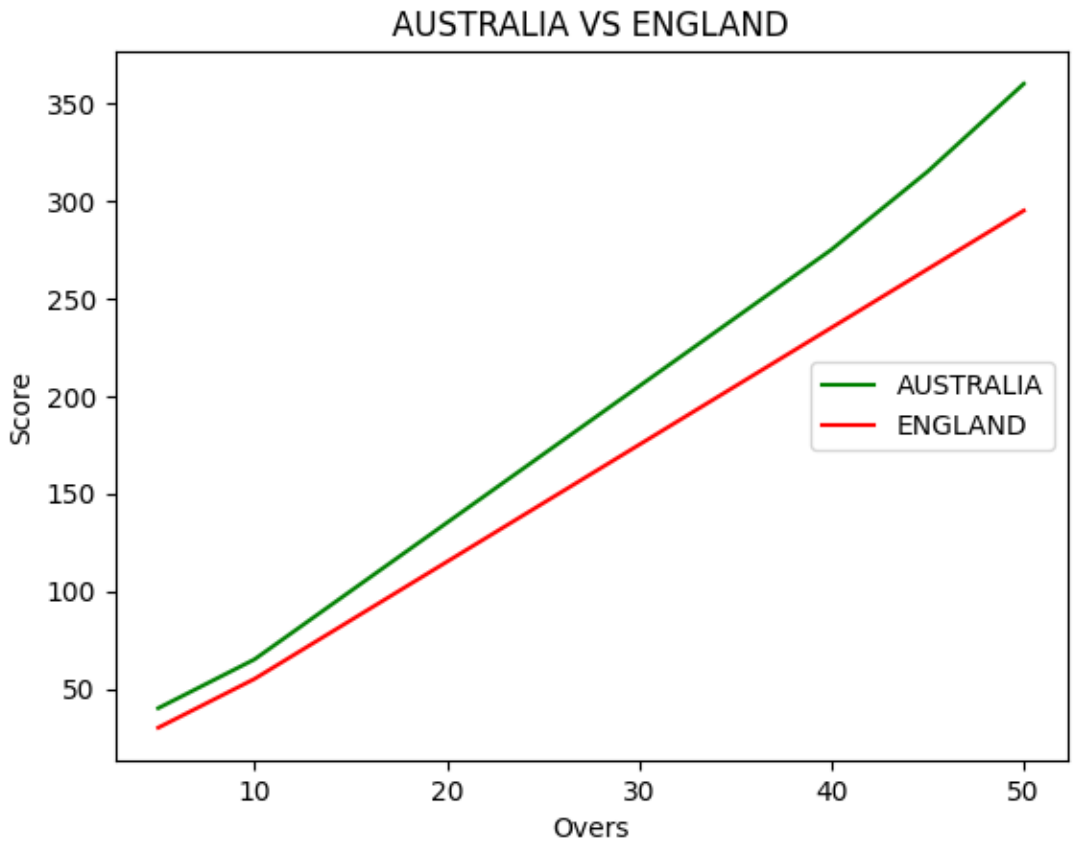
```
import matplotlib.pyplot as cricket

Overs=list(range(5,51,5))
Australia_Score=[40,65,100,135,170,205,240,275,315,360]
England_Score=[30,55,85,115,145,175,205,235,265,295]

cricket.plot(Overs,Australia_Score)
cricket.plot(Overs,England_Score)
cricket.show()
cricket.title("AUSTRALIA VS ENGLAND")
cricket.xlabel("Overs")
cricket.ylabel("Score")
cricket.legend()
cricket.plot(Overs,Australia_Score,color="green",label="AUSTRALIA")
cricket.plot(Overs,England_Score,color="red",label="ENGLAND")
cricket.legend(loc="center right")
```

**Output:**




**Result:**

Thus, the cricket data was successfully visualized using the chart.

**3.b)**

# Conduct an experiment to show data visualization using bar chart

**Description:** Take any sample data either through csv file or data fetched directly through code.

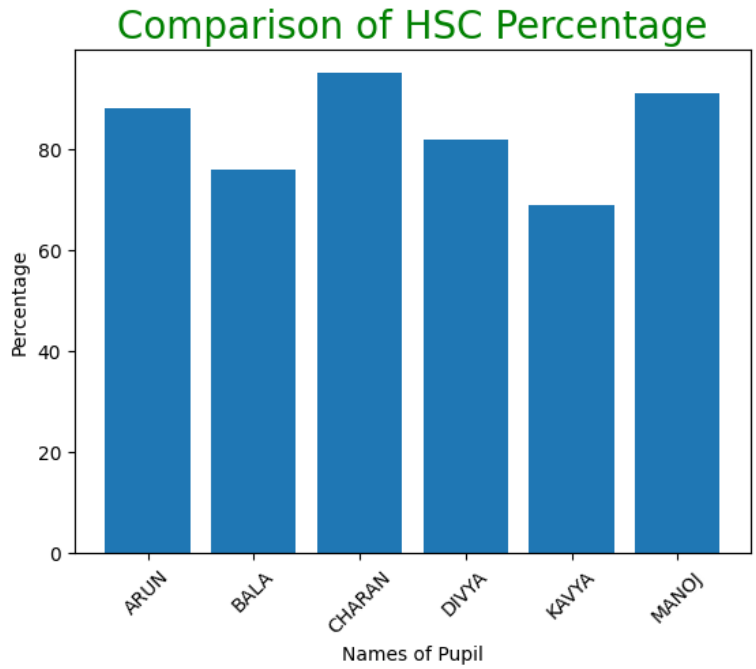
Program:

```
import matplotlib.pyplot as hsemark
import numpy as np

Names = ['ARUN','BALA','CHARAN','DIVYA','KAVYA','MANOJ']
xaxis = np.arange(len(Names))
Percentage_hsc = [88,76,95,82,69,91]

hsemark.bar(Names, Percentage_hsc)
hsemark.xticks(xaxis, Names, rotation=45)
hsemark.xlabel('Names of Pupil')
hsemark.ylabel('Percentage')
hsemark.title('Comparison of HSC Percentage', fontsize=20, color='green')
hsemark.show()
```

**Output:**




**Result:**

Thus, the bar chart was successfully created to compare the HSC percentages of different pupils.

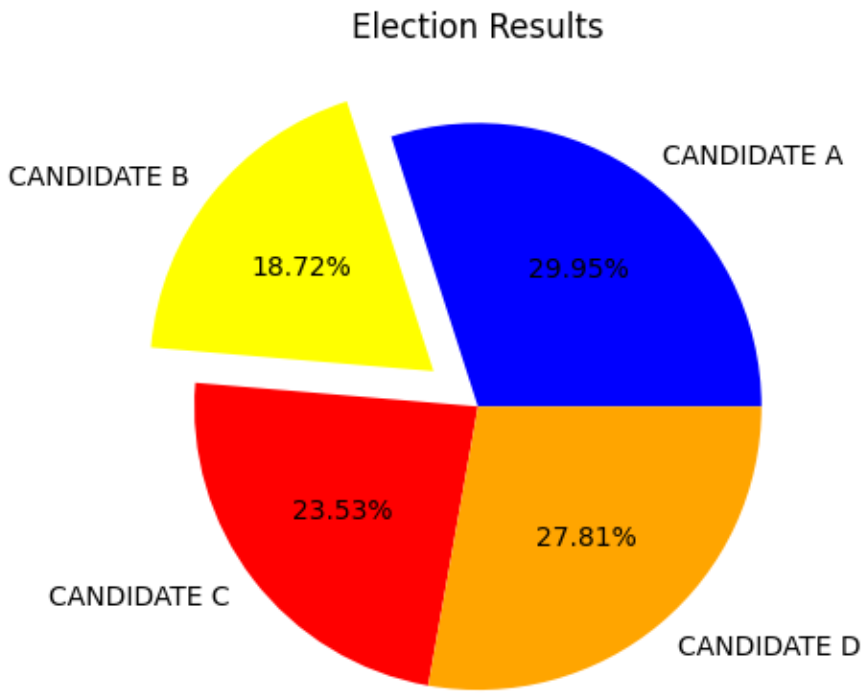
3.c)

# Conduct an experiment to show data visualization using pie chart

**Description:**
Take any sample data either through csv file or data fetched directly through code.
**Program:**

```
import matplotlib.pyplot as election
labels = ['CANDIDATE A','CANDIDATE B','CANDIDATE C','CANDIDATE D']
Votes = [280,175,220,260]
colors = ['blue','yellow','red','orange']
explode = (0,0.2,0,0)
election.pie(Votes, labels=labels, colors=colors, explode=explode, autopct='%0.2f%
%')
election.title('Election Results')
election.show()
```

**Output:**




**Result:**

Thus, the pie chart was successfully created to represent the election results of different candidates.

**EXPERIMENT NO:4**

## To Count the frequency of occurrence of a word in a body of text is often

## needed during text processing.

**Description:**

Import the word_tokenize function and gutenberg.
Program:

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import gutenberg
nltk.download('gutenberg')
nltk.download('punkt')
sample = gutenberg.raw("shakespeare-hamlet.txt")
token = word_tokenize(sample)
wlist = []
for i in range(40):
   wlist.append(token[i])
wordfreq = [wlist.count(w) for w in wlist]
print("Pairs\n" + str(list(zip(wlist, wordfreq))))
```

Output:

```
[nltk_data] Downloading package gutenberg to /root/nltk_data...
[nltk_data]   Unzipping corpora/gutenberg.zip.
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
Pairs
[('[', 1), ('The', 1), ('Tragedie', 1), ('of', 1), ('Hamlet', 1), ('by', 1), ('William', 1),
('Shakespeare', 1), ('1599', 1), (']', 1), ('Actus', 1), ('Primus', 1), ('.', 5), ('Scoena', 1),
('Prima', 1), ('.', 5), ('Enter', 1), ('Barnardo', 2), ('and', 1), ('Francisco', 1), ('two', 1),
('Centinels', 1), ('.', 5), ('Barnardo', 2), ('.', 5), ('Who', 1), ('"s", 1), ('there', 1), ('?', 1), ('Fran',
1), ('.', 5), ('Nay', 1), ('answer', 1), ('me', 1), (':', 1), ('Stand', 1), ('&', 1), ('vnfold', 1), ('your',
1), ('selfe', 1)]
```

Result:

Thus, the frequency of occurrence of words in the given text was successfully calculated and displayed using Python and NLTK.

## Exp No: 5 — Data Collection and Initial Exploration

### Aim

To collect, load, and perform initial exploration of the diabetes dataset.

### Description

The diabetes dataset is loaded using Pandas. The dataset is explored using head(), info() and describe(). Histograms are used to visualize the distribution of each feature and a pair plot is used to visualize relationships between the features.

### Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.data.csv"

columns = ['Pregnancies','Glucose','BloodPressure','SkinThickness',
        'Insulin','BMI','DiabetesPedigreeFunction','Age','Outcome']

db = pd.read_csv(url, names=columns)

print(db.head())

print(db.info())
print(db.describe())

db.hist(bins=50, figsize=(20,15))
plt.show()

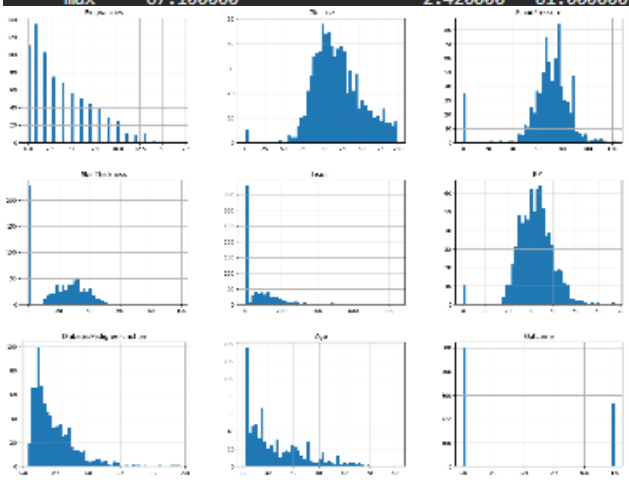
sns.pairplot(db)
plt.show()
```

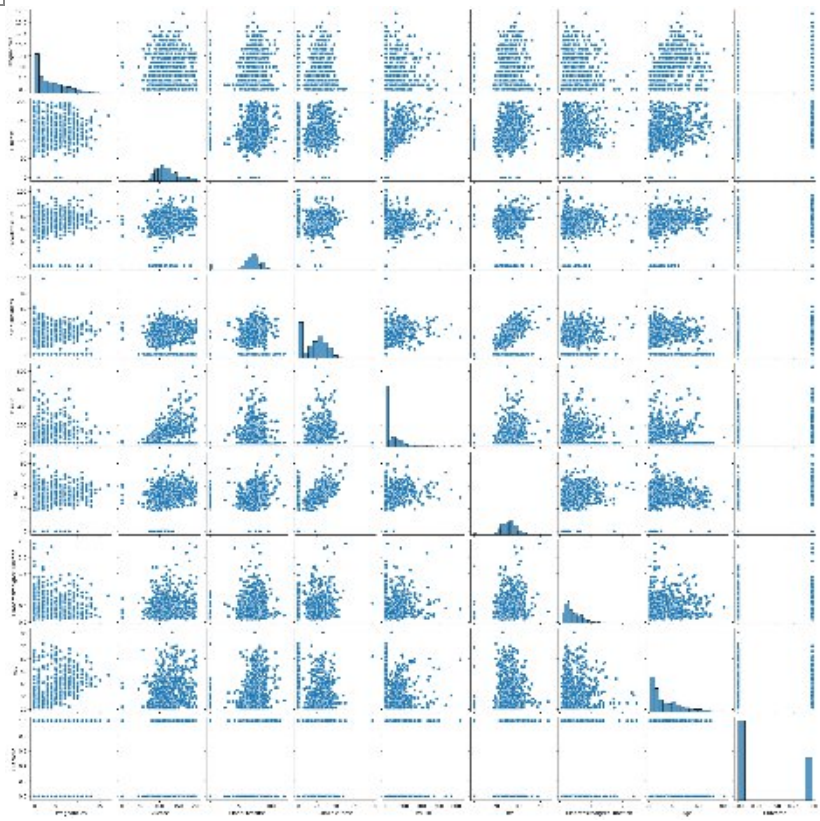
Output:

```
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
0            6      148             72             35        0  33.6
1            1       85             66             29        0  26.6
2            8      183             64              0        0  23.3
3            1       89             66             23       94  28.1
4            0      137             40             35      168  43.1

   DiabetesPedigreeFunction  Age  Outcome
0                     0.627   50        1
1                     0.351   31        0
2                     0.672   32        1
3                     0.167   21        0
4                     2.288   33        1
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Pregnancies               768 non-null    int64
 1   Glucose                   768 non-null    int64
 2   BloodPressure             768 non-null    int64
 3   SkinThickness             768 non-null    int64
 4   Insulin                   768 non-null    int64
 5   BMI                       768 non-null    float64
 6   DiabetesPedigreeFunction  768 non-null    float64
 7   Age                       768 non-null    int64
 8   Outcome                   768 non-null    int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
None
       Pregnancies     Glucose  BloodPressure  SkinThickness     Insulin  \
count   768.000000  768.000000     768.000000     768.000000  768.000000
mean      3.845052  120.894531      69.105469      20.536458   79.799479
std       3.369578   31.972618      19.355807      15.952218  115.244002
min       0.000000    0.000000       0.000000       0.000000    0.000000
25%       1.000000   99.000000      62.000000       0.000000    0.000000
50%       3.000000  117.000000      72.000000      23.000000   30.500000
75%       6.000000  140.250000      80.000000      32.000000  127.250000
max      17.000000  199.000000     122.000000      99.000000  846.000000

              BMI  DiabetesPedigreeFunction         Age     Outcome
count  768.000000                768.000000  768.000000  768.000000
mean    31.992578                  0.471876   33.240885    0.348958
std      7.884160                  0.331329   11.760232    0.476951
min      0.000000                  0.078000   21.000000    0.000000
25%     27.300000                  0.243750   24.000000    0.000000
50%     32.000000                  0.372500   29.000000    0.000000
75%     36.600000                  0.626250   41.000000    1.000000
max     67.100000                  2.420000   81.000000    1.000000
```





## Result

The diabetes dataset was successfully loaded and initially explored. The first few records, dataset structure and summary statistics were displayed. Histograms showed the distribution of each feature, and the pair plot showed the relationships between the variables.

# EXPERIMENT 6

## AIM:

To analyze daily website traffic data using time series techniques to identify trends, seasonality, and anomalies.

## CODE:

```
import pandas as pd

import matplotlib.pyplot as plt

from statsmodels.tsa.seasonal import seasonal_decompose


data = pd.read_csv("website_traffic_timeseries.csv", parse_dates=["Date"], index_col="Date")


data.interpolate(method="linear", inplace=True)


print(data.head())


print(data.info())

print(data.describe())



from statsmodels.tsa.seasonal import seasonal_decompose


decomposition = seasonal_decompose(

    data["Page_Views"],

    model="additive",
```

```
    period=7
)

decomposition.plot()

plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
# Moving Average
data["Page_Views_MA"] = data["Page_Views"].rolling(window=30).mean()

data[["Page_Views", "Page_Views_MA"]].plot(figsize=(12, 5))
plt.title("Page Views with Moving Average")
plt.xlabel("Date")
plt.ylabel("Page Views")
plt.show()

# Anomaly Detection using Z-score
data["Page_Views_Z"] = (
    (data["Page_Views"] - data["Page_Views"].mean())
    / data["Page_Views"].std()
)

anomalies = data[data["Page_Views_Z"].abs() > 3]
```

```
print("Anomalies:")
print(anomalies[["Page_Views", "Page_Views_Z"]])


plt.figure(figsize=(12, 5))
plt.plot(data.index, data["Page_Views"], label="Page Views")
plt.scatter(
    anomalies.index,
    anomalies["Page_Views"],
    color="red",
    label="Anomalies"
)
plt.title("Website Traffic Anomaly Detection")
plt.xlabel("Date")
plt.ylabel("Page Views")
plt.legend()
plt.show()
```

OUTPUT:

| | Page_Views | Unique_Visitors | Bounce_Rate |
|---|---|---|---|
| Date | | | |
| 2025-01-01 | 2074 | 1280 | 48.91 |
| 2025-01-02 | 2326 | 1306 | 44.98 |
| 2025-01-03 | 2999 | 1678 | 45.09 |
| 2025-01-04 | 3185 | 1863 | 45.93 |
| 2025-01-05 | 2586 | 1555 | 50.24 |

<class 'pandas.core.frame.DataFrame'>

DatetimeIndex: 365 entries, 2025-01-01 to 2025-12-31

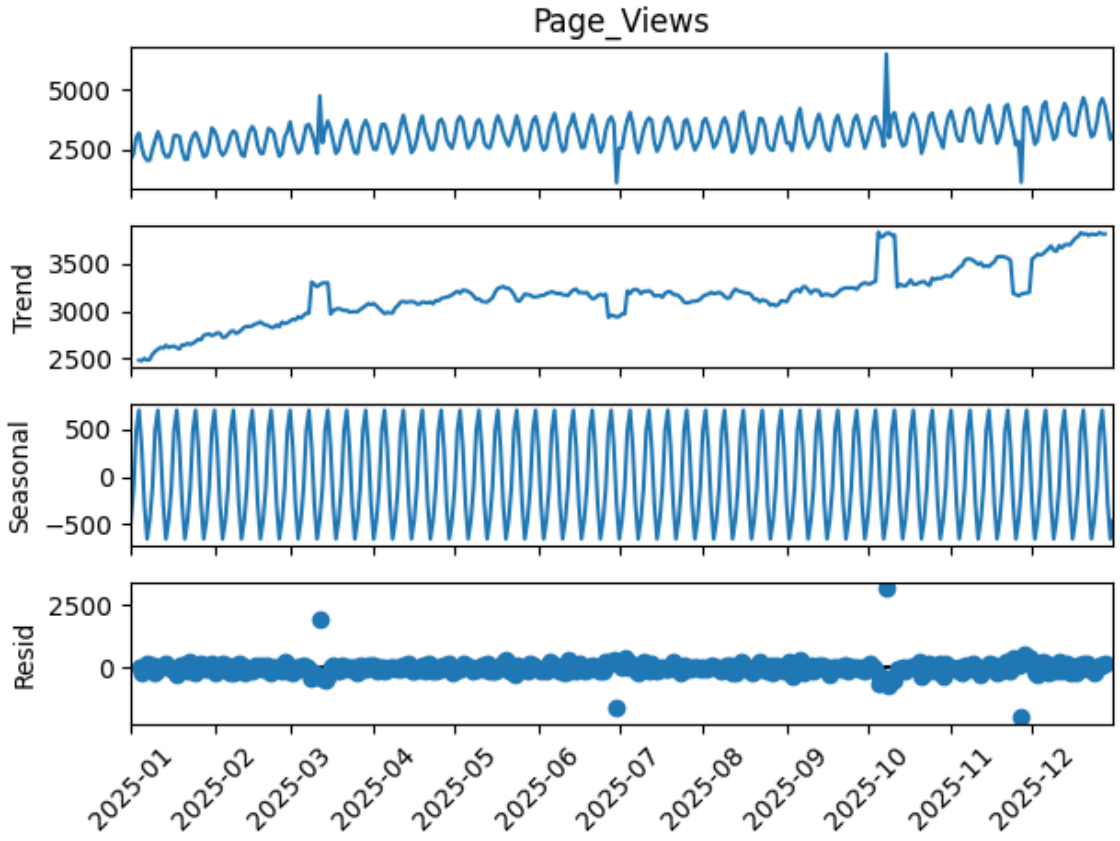
Data columns (total 3 columns):

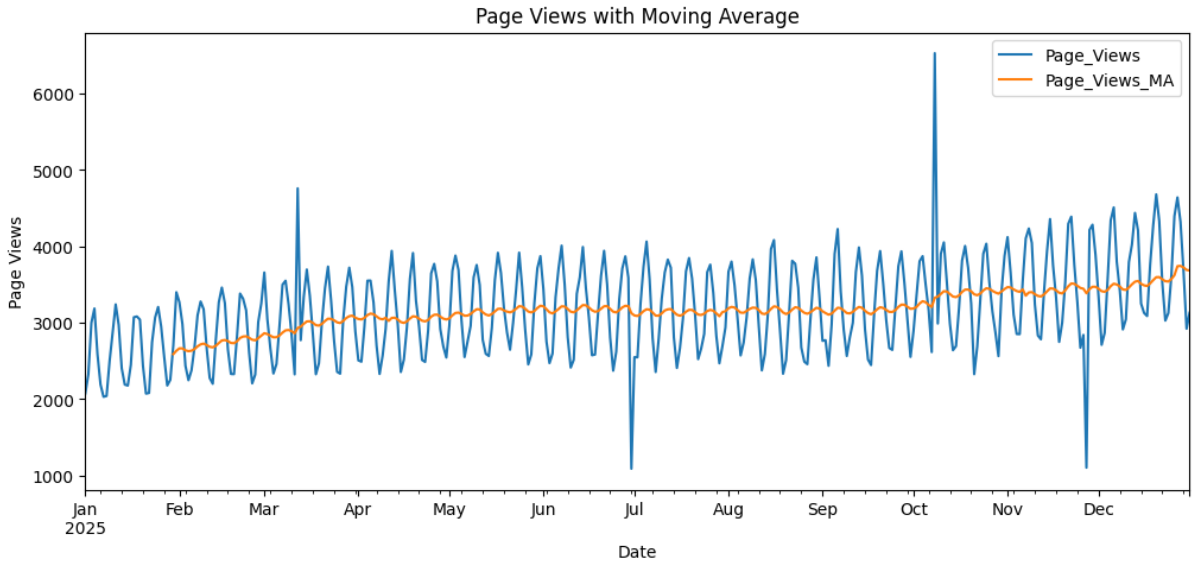
| # | Column | Non-Null Count | Dtype |
|---|---|---|---|
| 0 | Page_Views | 365 non-null | int64 |
| 1 | Unique_Visitors | 365 non-null | int64 |
| 2 | Bounce_Rate | 365 non-null | float64 |

dtypes: float64(1), int64(2)

memory usage: 11.4 KB

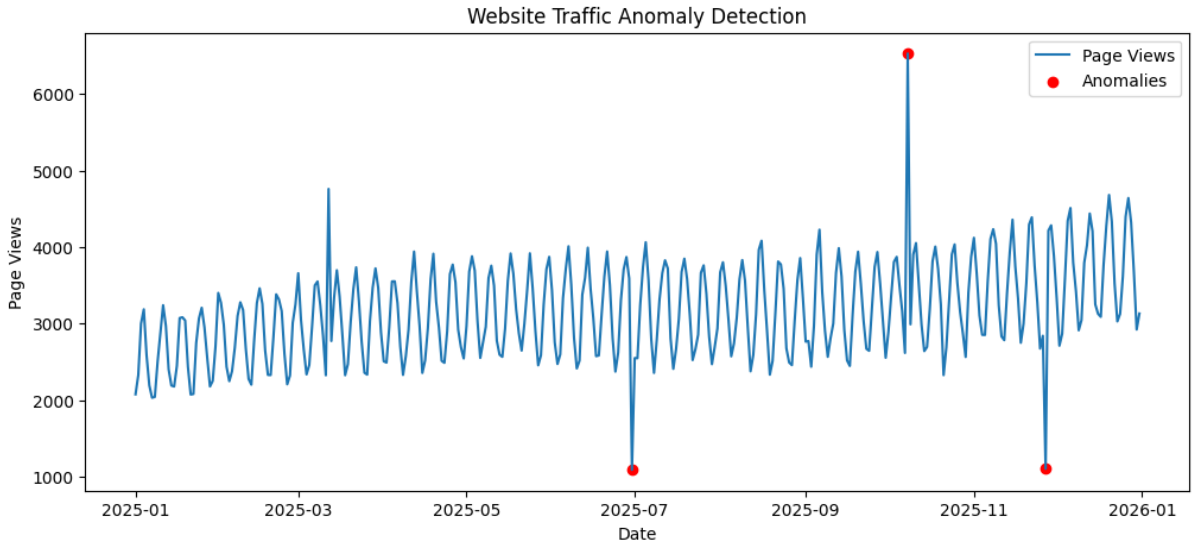
| | Page_Views | Unique_Visitors | Bounce_Rate |
|---|---|---|---|
| count | 365.000000 | 365.000000 | 365.000000 |
| mean | 3166.630137 | 1830.115068 | 48.036904 |
| std | 623.902005 | 371.898955 | 3.479127 |
| min | 1087.000000 | 617.000000 | 37.530000 |
| 25% | 2686.000000 | 1550.000000 | 45.310000 |
| 50% | 3105.000000 | 1797.000000 | 48.240000 |
| 75% | 3614.000000 | 2083.000000 | 50.680000 |
| max | 6529.000000 | 3764.000000 | 60.550000 |






Anomalies:

| Date | Page_Views | Page_Views_Z |
|---|---|---|
| 2025-06-30 | 1087 | -3.333264 |
| 2025-10-08 | 6529 | 5.389260 |
| 2025-11-27 | 1101 | -3.310825 |



## RESULT:

The website traffic data was successfully analyzed, and trends, seasonal patterns, moving averages, and anomalies were identified.

# EXPERIMENT 7

## AIM:

To explore random sampling and understand the concept of sampling distribution using Python.

## CODE:

```
import numpy as np
import matplotlib.pyplot as plt


# Step 1: Generate a population
population_mean = 100
population_std = 15
population_size = 50000


population = np.random.normal(
    population_mean,
    population_std,
    population_size
)


# Step 2: Random sampling
sample_sizes = [25, 50, 100]
num_samples = 500


sample_means = {}
```

```
for size in sample_sizes:
    sample_means[size] = [ ]

    for _ in range(num_samples):
        sample = np.random.choice(
            population,
            size=size,
            replace=False
        )
        sample_means[size].append(np.mean(sample))

# Step 3: Plot sampling distributions
plt.figure(figsize=(12, 8))

for i, size in enumerate(sample_sizes):
    plt.subplot(3, 1, i + 1)

    plt.hist(
        sample_means[size],
        bins=30,
        alpha=0.7,
        label=f'Sample Size {size}'
    )

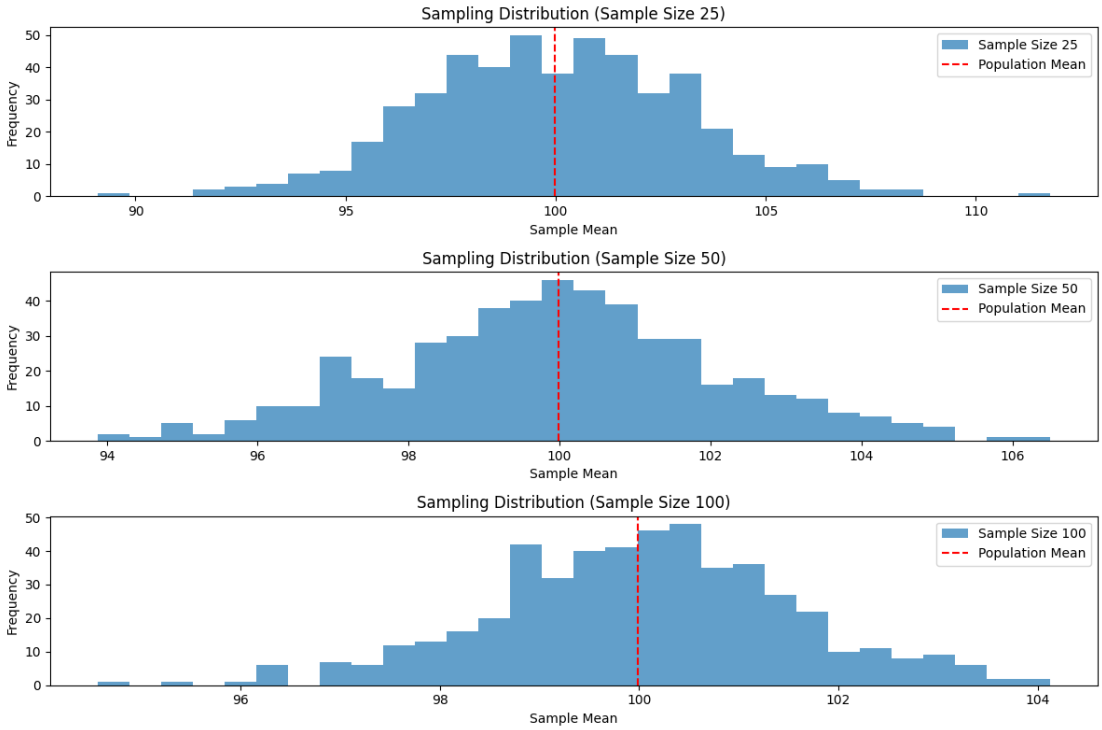
    plt.axvline(
```

```
        np.mean(population),
        color='red',
        linestyle='--',
        linewidth=1.5,
        label='Population Mean'
    )

    plt.title(f'Sampling Distribution (Sample Size {size})')
    plt.xlabel('Sample Mean')
    plt.ylabel('Frequency')
    plt.legend()

plt.tight_layout()
plt.show()
```

## OUTPUT:




## RESULT:

Random samples and their sampling distributions were successfully generated. The results showed that larger sample sizes produce a narrower sampling distribution around the population mean.

# EXPERIMENT 8

## AIM:

To perform a **two-tailed Z-test** on the salary dataset to determine whether the average salary differs significantly from the specified population mean.

## CODE:

```
import pandas as pd

import numpy as np

import scipy.stats as stats


data = pd.read_excel("Salary_data.xlsx")


sample_data = data["Salary"].values


population_mean = 70000


sample_mean = np.mean(sample_data)

sample_std = np.std(sample_data, ddof=1)


n = len(sample_data)


z_statistic = (sample_mean - population_mean) / (sample_std / np.sqrt(n))


p_value = 2 * (1 - stats.norm.cdf(abs(z_statistic)))


print(f"Sample Mean: {sample_mean:.2f}")
```

```
print(f"Z-Statistic: {z_statistic:.4f}")
print(f"P-Value: {p_value:.4f}")

alpha = 0.05

if p_value < alpha:
    print("Reject the null hypothesis: The average salary is significantly different from ₹70,000.")
else:
    print("Fail to reject the null hypothesis: There is no significant difference in average salary from ₹70,000.")
```

## OUTPUT:

```
Sample Mean: 76003.00
Z-Statistic: 1.1994
P-Value: 0.2304
Fail to reject the null hypothesis: There is no significant difference in average salary from ₹70,000.
```

## RESULT:

The Z-test was successfully performed, and the result showed no significant difference between the average salary and the specified population mean.

# EXPERIMENT 9

## AIM:

To understand the use of the Matplotlib library in Data Science through visualization.

## CODE:

```
import pandas as pd

import numpy as np

import seaborn as sns

import matplotlib.pyplot as plt


# Load dataset

data = pd.read_excel('/content/Social_Network_Ads.xlsx')

data


# Dataset information

data.info()


# Summary statistics

data.describe()


# Count values of Gender

data.value_counts('Gender')


# Count plot

sns.countplot(x='Gender', data=data)
```

```
plt.show()

# Convert Gender into dummy variables
dummies = pd.get_dummies(data.Gender)

FinalDataset = pd.concat(
    [dummies, data.iloc[:, [0, 2, 3, 4]]],
    axis=1
)

FinalDataset.head()

# Scatter plot 1
sns.scatterplot(
    x='Age',
    y='EstimatedSalary',
    hue='Gender',
    data=data
)
plt.show()

# Scatter plot 2
sns.scatterplot(
    x='Age',
    y='EstimatedSalary',
```

```
    hue='Purchased',
    data=data
)
plt.show()

# Pairplot
g = sns.pairplot(
    data[['Age', 'EstimatedSalary', 'Purchased']],
    hue='Purchased',
    height=3
)

# Rotate x-axis labels on all subplots
for ax in g.axes.flat:
    if ax is not None:
        ax.tick_params(axis='x', labelrotation=45)

plt.show()

# Histogram 1 - Estimated Salary
sns.FacetGrid(
    data,
    hue='Gender',
    height=5
).map(
```

```
    sns.histplot,
    'EstimatedSalary'
).add_legend()

plt.show()

# Histogram 2 - Age
sns.FacetGrid(
    data,
    hue='Gender',
    height=5
).map(
    sns.histplot,
    'Age'
).add_legend()

plt.show()
```

OUTPUT:

| | User ID | Gender | Age | EstimatedSalary | Purchased |
|---|---|---|---|---|---|
| **0** | 15624510 | Male | 19 | 19000 | 0 |
| **1** | 15810944 | Male | 35 | 20000 | 0 |
| **2** | 15668575 | Female | 26 | 43000 | 0 |
| **3** | 15603246 | Female | 27 | 57000 | 0 |
| **4** | 15804002 | Male | 19 | 76000 | 0 |
| **...** | ... | ... | ... | ... | ... |
| **395** | 15691863 | Female | 46 | 41000 | 1 |
| **396** | 15706071 | Male | 51 | 23000 | 1 |
| **397** | 15654296 | Female | 50 | 20000 | 1 |
| **398** | 15755018 | Male | 36 | 33000 | 0 |
| **399** | 15594041 | Female | 49 | 36000 | 1 |

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 400 entries, 0 to 399

Data columns (total 5 columns):

| # | Column | Non-Null Count | Dtype |
|---|---|---|---|
| 0 | User ID | 400 non-null | int64 |
| 1 | Gender | 400 non-null | object |
| 2 | Age | 400 non-null | int64 |
| 3 | EstimatedSalary | 400 non-null | int64 |
| 4 | Purchased | 400 non-null | int64 |

dtypes: int64(4), object(1)

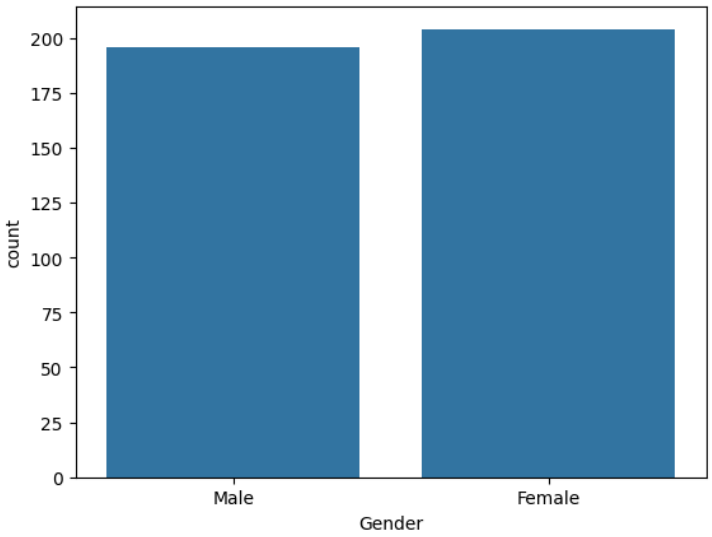
memory usage: 15.8+ KB

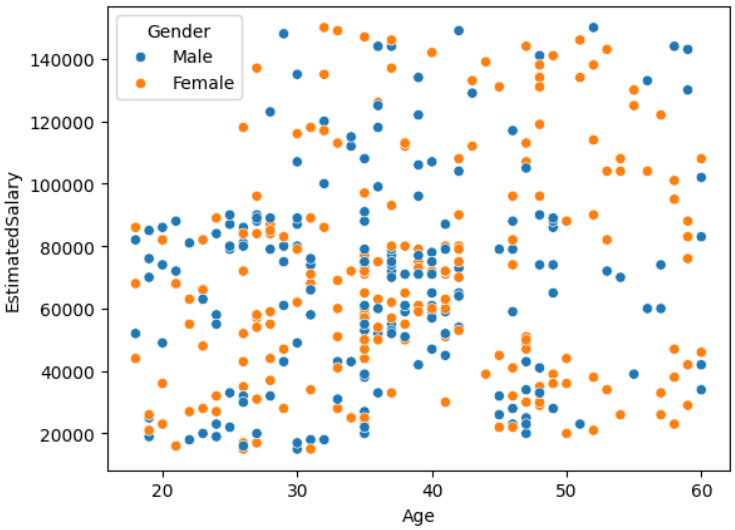
| | User ID | Age | EstimatedSalary | Purchased |
|---|---|---|---|---|
| **count** | 4.000000e+02 | 400.000000 | 400.000000 | 400.000000 |
| **mean** | 1.569154e+07 | 37.655000 | 69742.500000 | 0.357500 |
| **std** | 7.165832e+04 | 10.482877 | 34096.960282 | 0.479864 |
| **min** | 1.556669e+07 | 18.000000 | 15000.000000 | 0.000000 |
| **25%** | 1.562676e+07 | 29.750000 | 43000.000000 | 0.000000 |
| **50%** | 1.569434e+07 | 37.000000 | 70000.000000 | 0.000000 |
| **75%** | 1.575036e+07 | 46.000000 | 88000.000000 | 1.000000 |
| **max** | 1.581524e+07 | 60.000000 | 150000.000000 | 1.000000 |

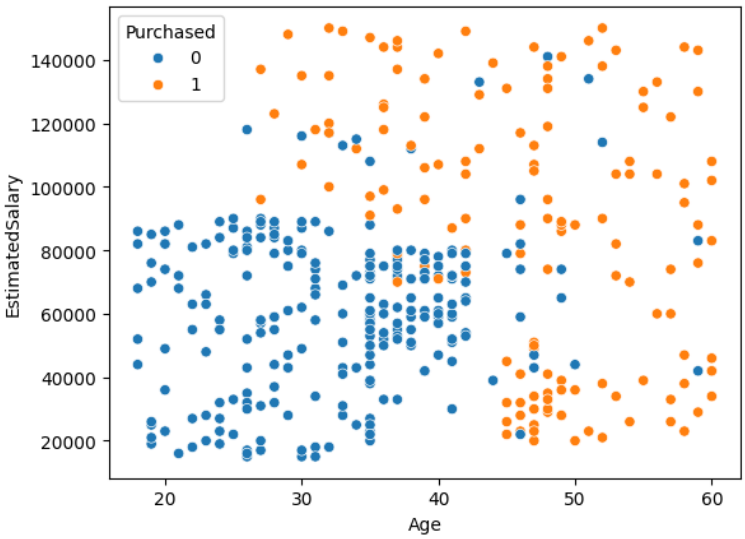
| | count |
|---|---|
| **Gender** | |
| **Female** | 204 |
| **Male** | 196 |

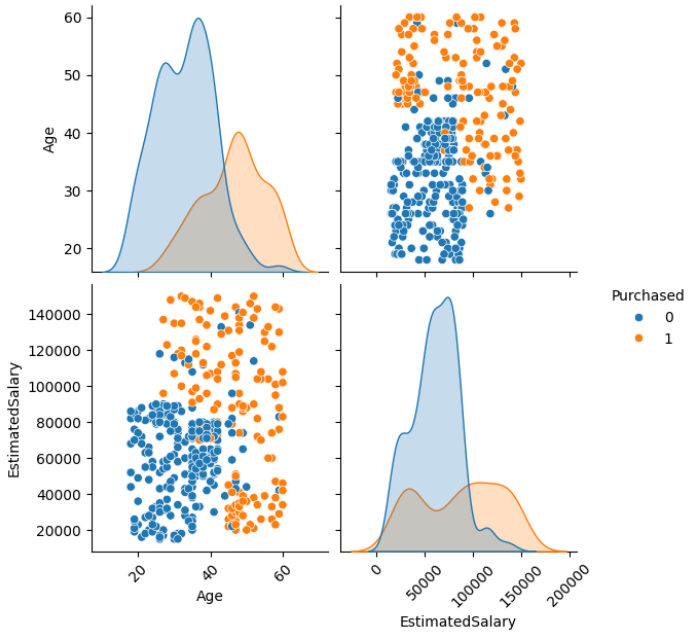
**dtype:** int64

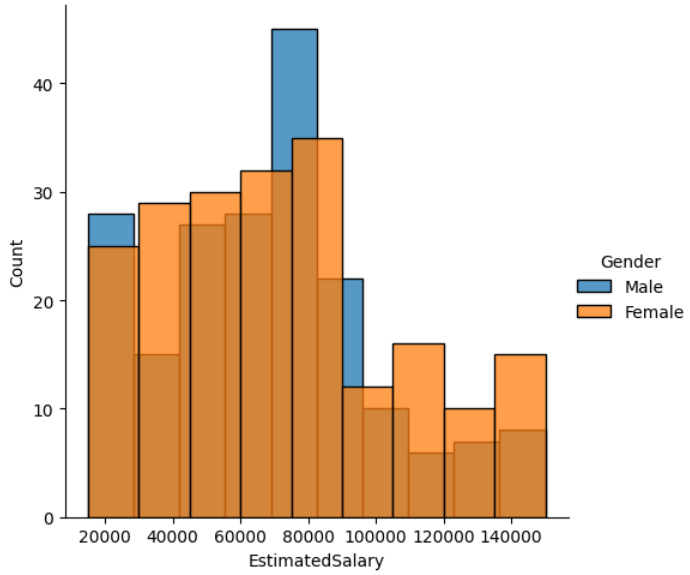
| | Female | Male | User ID | Age | EstimatedSalary | Purchased |
|---|---|---|---|---|---|---|
| **0** | False | True | 15624510 | 19 | 19000 | 0 |
| **1** | False | True | 15810944 | 35 | 20000 | 0 |
| **2** | True | False | 15668575 | 26 | 43000 | 0 |
| **3** | True | False | 15603246 | 27 | 57000 | 0 |
| **4** | False | True | 15804002 | 19 | 76000 | 0 |

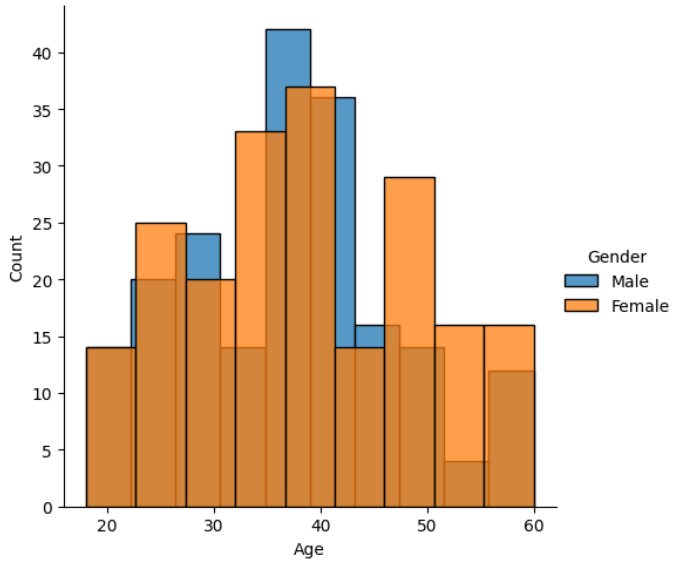

count
200
175
150
125
100
75
50
25
0
Male
Female
Gender


Gender
Male
Female
EstimatedSalary
140000
120000
100000
80000
60000
40000
20000
20
30
40
50
60
Age


Purchased
0
1
EstimatedSalary
140000
120000
100000
80000
60000
40000
20000
20
30
40
50
60
Age


Age
EstimatedSalary
Purchased
0
1
20
40
60
20000
40000
60000
80000
100000
120000
140000
0
50000
100000
150000
200000


Count
EstimatedSalary
Gender
Male
Female
0
10
20
30
40
20000
40000
60000
80000
100000
120000
140000


Count
Age
Gender
Male
Female
0
5
10
15
20
25
30
35
40
20
30
40
50
60

## RESULT:

The Iris dataset was successfully visualized using different plots, helping to understand the distribution and relationships among the data features.

# EXPERIMENT 10

## AIM:

To understand and perform basic array functions using the NumPy library in Python.

## CODE:

```
import numpy as np


# Create 1-D array using values from the dataset

array = np.array([15, 15, 16, 16, 17, 17, 18, 18, 19])


print("1. Original Array:")

print(array)


# Square root

print("\n2. Square Root of Array:")

print(np.sqrt(array))


# Number of dimensions

print("\n3. Number of Dimensions:")

print(array.ndim)


# Reshape into 3x3

new_array = array.reshape(3, 3)


print("\n4. Reshaped Array:")
```

```
print(new_array)

# Number of dimensions after reshape
print("\n5. Number of Dimensions after Reshape:")
print(new_array.ndim)

# Ravel
print("\n6. Ravelled Array:")
print(new_array.ravel())

# Reshape again
newm = new_array.reshape(3, 3)

print("\n7. New Array:")
print(newm)

# Slicing operations
print("\n8. newm[2,1:3]:")
print(newm[2,1:3])

print("\n9. newm[1:2,1:3]:")
print(newm[1:2,1:3])

print("\n10. new_array[0:3,0:0]:")
print(new_array[0:3,0:0])
```

```
print("\n11. new_array[0:2,0:1]:")
print(new_array[0:2,0:1])

print("\n12. new_array[0:3,0:1]:")
print(new_array[0:3,0:1])

print("\n13. new_array[1:3]:")
print(new_array[1:3])
```

OUTPUT:

```
1. Original Array:
[15 15 16 16 17 17 18 18 19]

2. Square Root of Array:
[3.87298335 3.87298335 4.         4.         4.12310563 4.12310563
 4.24264069 4.24264069 4.35889894]

3. Number of Dimensions:
1

4. Reshaped Array:
[[15 15 16]
```

[16 17 17]

[18 18 19]]

5. Number of Dimensions after Reshape:

2

6. Ravelled Array:

[15 15 16 16 17 17 18 18 19]

7. New Array:

[[15 15 16]

[16 17 17]

[18 18 19]]

8. newm[2,1:3]:

[18 19]

9. newm[1:2,1:3]:

[[17 17]]

10. new_array[0:3,0:0]:

[]

11. new_array[0:2,0:1]:

[[15]

[16]]

12. new_array[0:3,0:1]:

[[15]

[16]

[18]]

13. new_array[1:3]:

[[16 17 17]

[18 18 19]]

RESULT:

The NumPy array functions were successfully performed, and the array dimensions, reshaped arrays, and slicing operations were obtained.

# Exp No:11 Experiment to understand pandas library use cases in Data science.

## Aim:

To understand the use of the Pandas library in Data Science by creating and manipulating DataFrames, reading CSV files, and performing basic statistical operations on employee data.

## Program Description:

This program demonstrates the use of Pandas DataFrame for handling tabular data. It creates a DataFrame using a list, assigns column names, reads employee data from CSV files, and displays the first and last records using head() and tail(). The program also uses info() to obtain information about the dataset and performs statistical operations such as mean, median, mode, variance, standard deviation, and descriptive statistics using describe(). Finally, the DataFrame columns and values are accessed and a new DataFrame is created from the extracted values.

## Program:

```
import numpy as np

import pandas as pd
```

Edit with WPS Office

```
# Create DataFrame
list1=[[1,'Arun',85],[2,'Priya',92]]
df=pd.DataFrame(list1)
df.columns=["ID","Name","Marks"]
df
df.info()

# --- ADDED THIS LINE TO GENERATE THE FILE SO IT DOES NOT FAIL ---
df.to_csv("Students.csv", index=False)

# Read CSV (Replaces the above df)
df=pd.read_csv("Students.csv")
df
df.info()
df.head()
df.tail()

# Column Operations
df["Marks"]
type(df["Marks"])
df["Marks"].mean()
```

Edit with WPS Office

```
df["Marks"].median()
df["Marks"].mode()
df["Marks"].var()
df["Marks"].std()

# Descriptions
df.describe()
df.describe(include='all')

# Array Conversion & Rebuilding
empcol=df.columns
empcol
emparray=df.values
emparray
Student_df=pd.DataFrame(emparray,columns=empcol)
Student_df
```

OUTPUT:

Edit with WPS Office

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2 entries, 0 to 1
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   ID      2 non-null      int64
 1   Name    2 non-null      object
 2   Marks   2 non-null      int64
dtypes: int64(2), object(1)
memory usage: 180.0+ bytes
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2 entries, 0 to 1
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   ID      2 non-null      int64
 1   Name    2 non-null      object
 2   Marks   2 non-null      int64
dtypes: int64(2), object(1)
memory usage: 180.0+ bytes
```

| | ID | Name | Marks |
|---|---|---|---|
| 0 | 1 | Arun | 85 |
| 1 | 2 | Priya | 92 |

Result:

Thus, the Pandas library was successfully used to create, read, analyze, and manipulate DataFrames. Basic statistical measures and information about the employee dataset were obtained successfully.

Edit with WPS Office

Ex:no:12

Experiment to detect outliers in a given data set.

AIM:

To identify and remove outliers from a given dataset using the **IQR (Interquartile Range) method**.

PROGRAM DESCRIPTION:

The program generates a random dataset and calculates the **25th, 50th and 75th percentiles**. The lower range (LR) and upper range (UR) are calculated using IQR. Values outside these ranges are identified as outliers and removed from the dataset.

PROGRAM:

```
import numpy as np

import matplotlib.pyplot as plt

array=np.random.randint(10,120,18)

array

array.mean()

np.percentile(array,25)

np.percentile(array,50)

np.percentile(array,75)

np.percentile(array,100)

#outliers detection

def outDetection(array):

  sorted(array)

  Q1,Q3=np.percentile(array,[25,75])

  IQR=Q3-Q1

  lr=Q1-(1.5*IQR)

  ur=Q3+(1.5*IQR)

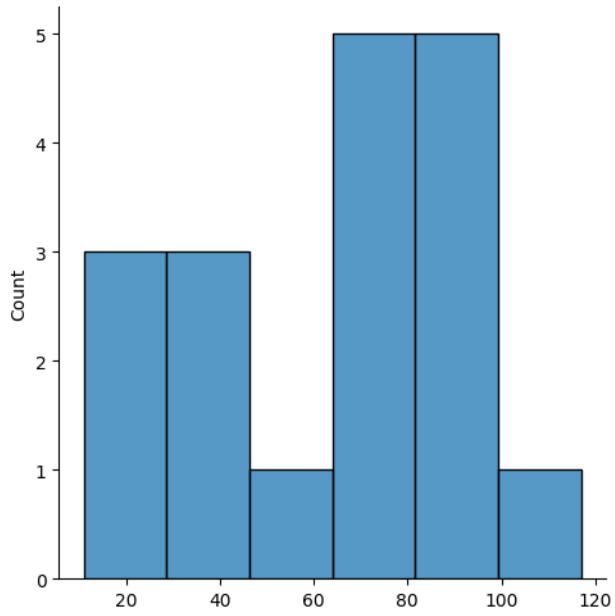
  return lr,ur

lr,ur=outDetection(array)

lr,ur
```

```
import seaborn as sns
%matplotlib inline
sns.displot(array)
plt.show()
sns.distplot(array)
plt.show()
new_array=array[(array>lr) & (array<ur)]
new_array
sns.displot(new_array)
plt.show()
lr1,ur1=outDetection(new_array)
lr1,ur1
final_array=new_array[(new_array>lr1) & (new_array<ur1)]
final_array
sns.distplot(final_array)
plt.show()
```

OUTPUT:



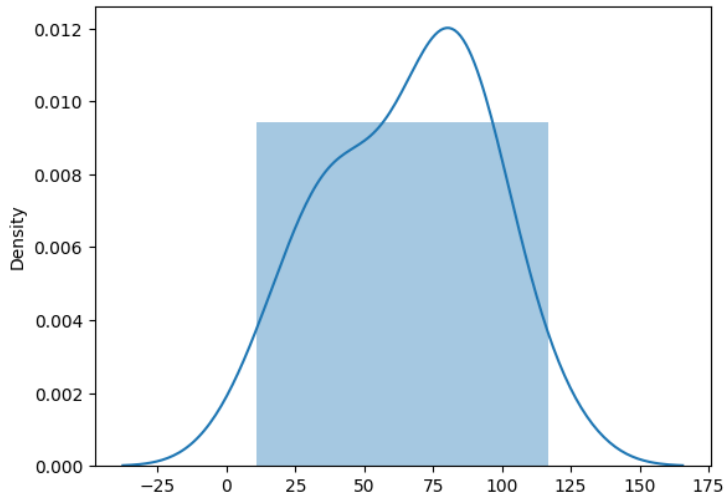

/tmp/ipykernel_608/3911722544.py:31: UserWarning:

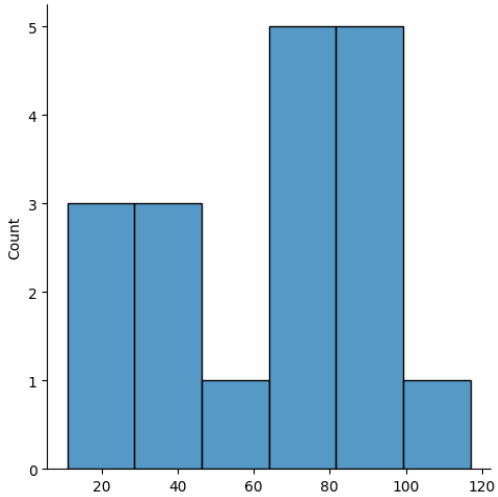
`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751

```
sns.distplot(array)
```




/tmp/ipykernel_608/3911722544.py:46: UserWarning:

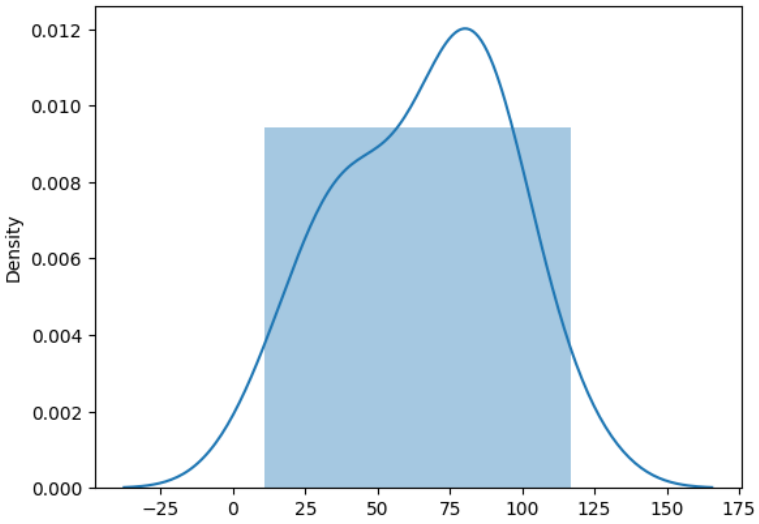
`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see

https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751

sns.distplot(final_array)




RESULT:

Thus, the outliers in the given dataset were successfully detected and removed using the **IQR method**, and the final dataset was obtained.

Exp No:13 Experiment to handle missing values and inappropriate data in a given data set

Aim:

To demonstrate the handling of missing and inappropriate data in a dataset using Python Pandas for data preprocessing.

Program Description:

This program creates a dataset using Pandas and performs data preprocessing operations such as identifying duplicate records, removing duplicates, resetting the index, removing unnecessary columns, identifying missing values, finding unique values, and replacing missing numerical values using suitable statistical methods such as mean and median.

Program:

```
import numpy as np

import pandas as pd

# Create the dataset

data = {

    "CustomerID": [1,2,3,4,5,6,7,8,9,10],

    "Age_Group": ["20-25","25-30","30-35","20-25","35+",

                  "35+","20-25","25-30","30-35","30-35"],

    "Rating(1-5)": [4,5,3,4,np.nan,5,4,2,5,4],

    "Hotel": ["Ibis","LemonTree","RedFox","LemonTree","Ibis",

              "Ibis","RedFox","LemonTree","Ibis","RedFox"],

    "FoodPreference": ["Veg","Non-Veg","Veg","Veg","Non-Veg",

                       "Veg","Non-Veg","Veg","Non-Veg","Non-Veg"],

    "Bill": [1500,2100,1350,np.nan,950,1800,1100,2400,3200,1750],

    "NoOfPax": [2,3,2,2,2,3,np.nan,3,4,2],

    "EstimatedSalary": [42000,56000,32000,60000,48000,

                        np.nan,35000,68000,91000,52000]

}
```

Edit with WPS Office

```
df = pd.DataFrame(data)
# Display dataset
print(df)
# The duplicated() method returns a Series with True and False
# values that describe which rows in the DataFrame are duplicated
df.duplicated()
print("\nDuplicate Values:")
print(df.duplicated())
# The info() method prints information about the DataFrame
df.info()
# Remove duplicate rows
df.drop_duplicates(inplace=True)
print("\nAfter Removing Duplicates:")
print(df)
# While removing duplicate record row index also removed
# The len() function returns the number of rows
print("\nLength of DataFrame:")
print(len(df))
# Reset the index
index = np.array(list(range(0, len(df))))
df.set_index(index, inplace=True)
print("\nNew Index:")
print(index)
print("\nDataFrame after Resetting Index:")
print(df)

# Axis refers to dimensions of DataFrame
# Drop unnecessary column if available
```


Edit with WPS Office

```
# Here we create an extra column for demonstration

df["Age_Group.1"] = df["Age_Group"]

print("\nBefore Dropping Age_Group.1:")
print(df)

df.drop(["Age_Group.1"], axis=1, inplace=True)

print("\nAfter Dropping Age_Group.1:")
print(df)

# loc is used for label indexing and accessing multiple columns

print("\nMissing CustomerID:")
print(df.loc[df["CustomerID"].isna()])

print("\nMissing Bill:")
print(df.loc[df["Bill"].isna()])

print("\nMissing EstimatedSalary:")
print(df.loc[df["EstimatedSalary"].isna()])

# Find missing values
print("\nMissing Values Count:")
print(df.isna().sum())

# Display unique values
```

Edit with WPS Office

```
print("\nUnique Age Groups:")
print(df["Age_Group"].unique())

print("\nUnique Hotels:")
print(df["Hotel"].unique())

# Display unique Food Preference
print("\nUnique Food Preferences:")
print(df["FoodPreference"].unique())

# Replace inappropriate value if present
df["FoodPreference"].replace(
    ["Vegetarian", "Veg"], ["Veg", "Veg"], inplace=True
)

print("\nFood Preference after Replacement:")
print(df["FoodPreference"])

# Fill missing values

# Numerical continuous data - use mean
df["EstimatedSalary"].fillna(
    round(df["EstimatedSalary"].mean()), inplace=True
)

# Numerical discrete data - use median
df["NoOfPax"].fillna(
    df["NoOfPax"].median(), inplace=True
```

Edit with WPS Office

```
)

# Numerical data - use median
df["Rating(1-5)"].fillna(
    round(df["Rating(1-5)"].median()), inplace=True
)

# Numerical continuous data - use mean
df["Bill"].fillna(
    round(df["Bill"].mean()), inplace=True
)

# Final cleaned dataset
print("\nFinal Cleaned Dataset:")
print(df)
```

Edit with WPS Office

```
   CustomerID Age_Group  Rating(1-5)      Hotel FoodPreference    Bill  \
0           1     20-25          4.0       Ibis            Veg  1500.0
1           2     25-30          5.0  LemonTree        Non-Veg  2100.0
2           3     30-35          3.0     RedFox            Veg  1350.0
3           4     20-25          4.0  LemonTree            Veg     NaN
4           5       35+          NaN       Ibis        Non-Veg   950.0
5           6       35+          5.0       Ibis            Veg  1800.0
6           7     20-25          4.0     RedFox        Non-Veg  1100.0
7           8     25-30          2.0  LemonTree            Veg  2400.0
8           9     30-35          5.0       Ibis        Non-Veg  3200.0
9          10     30-35          4.0     RedFox        Non-Veg  1750.0

   NoOfPax  EstimatedSalary
0      2.0          42000.0
1      3.0          56000.0
2      2.0          32000.0
3      2.0          60000.0
4      2.0          48000.0
5      3.0              NaN
6      NaN          35000.0
7      3.0          68000.0
8      4.0          91000.0
9      2.0          52000.0

Duplicate Values:
0    False
1    False
2    False
3    False
4    False
5    False
6    False
7    False
8    False
9    False
dtype: bool
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 8 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   CustomerID       10 non-null     int64
 1   Age_Group        10 non-null     object
 2   Rating(1-5)      9 non-null      float64
 3   Hotel            10 non-null     object
 4   FoodPreference   10 non-null     object
 5   Bill             9 non-null      float64
 6   NoOfPax          9 non-null      float64
 7   EstimatedSalary  9 non-null      float64
dtypes: float64(4), int64(1), object(3)
memory usage: 772.0+ bytes

After Removing Duplicates:
   CustomerID Age_Group  Rating(1-5)      Hotel FoodPreference    Bill  \
0           1     20-25          4.0       Ibis            Veg  1500.0
1           2     25-30          5.0  LemonTree        Non-Veg  2100.0
2           3     30-35          3.0     RedFox            Veg  1350.0
3           4     20-25          4.0  LemonTree            Veg     NaN
```

Edit with WPS Office

```
4           5       35+          NaN       Ibis        Non-Veg   950.0
5           6       35+          5.0       Ibis           Veg   1800.0
6           7     20-25          4.0     RedFox        Non-Veg  1100.0
7           8     25-30          2.0  LemonTree           Veg   2400.0
8           9     30-35          5.0       Ibis        Non-Veg  3200.0
9          10     30-35          4.0     RedFox        Non-Veg  1750.0

   NoOfPax  EstimatedSalary
0      2.0          42000.0
1      3.0          56000.0
2      2.0          32000.0
3      2.0          60000.0
4      2.0          48000.0
5      3.0              NaN
6      NaN          35000.0
7      3.0          68000.0
8      4.0          91000.0
9      2.0          52000.0

Length of DataFrame:
10

New Index:
[0 1 2 3 4 5 6 7 8 9]

DataFrame after Resetting Index:
   CustomerID Age_Group  Rating(1-5)      Hotel FoodPreference    Bill  \
```

```
0           1     20-25          4.0       Ibis           Veg   1500.0
1           2     25-30          5.0  LemonTree        Non-Veg  2100.0
2           3     30-35          3.0     RedFox           Veg   1350.0
3           4     20-25          4.0  LemonTree           Veg      NaN
4           5       35+          NaN       Ibis        Non-Veg   950.0
5           6       35+          5.0       Ibis           Veg   1800.0
6           7     20-25          4.0     RedFox        Non-Veg  1100.0
7           8     25-30          2.0  LemonTree           Veg   2400.0
8           9     30-35          5.0       Ibis        Non-Veg  3200.0
9          10     30-35          4.0     RedFox        Non-Veg  1750.0

   NoOfPax  EstimatedSalary
0      2.0          42000.0
1      3.0          56000.0
2      2.0          32000.0
3      2.0          60000.0
4      2.0          48000.0
5      3.0              NaN
6      NaN          35000.0
7      3.0          68000.0
8      4.0          91000.0
9      2.0          52000.0

Before Dropping Age_Group.1:
   CustomerID Age_Group  Rating(1-5)      Hotel FoodPreference    Bill  \
0           1     20-25          4.0       Ibis           Veg   1500.0
1           2     25-30          5.0  LemonTree        Non-Veg  2100.0
```

Edit with WPS Office

```
2           3     30-35          3.0     RedFox            Veg  1350.0
3           4     20-25          4.0  LemonTree            Veg     NaN
4           5       35+          NaN       Ibis        Non-Veg   950.0
5           6       35+          5.0       Ibis            Veg  1800.0
6           7     20-25          4.0     RedFox        Non-Veg  1100.0
7           8     25-30          2.0  LemonTree            Veg  2400.0
8           9     30-35          5.0       Ibis        Non-Veg  3200.0
9          10     30-35          4.0     RedFox        Non-Veg  1750.0

   NoOfPax  EstimatedSalary Age_Group.1
0      2.0          42000.0       20-25
1      3.0          56000.0       25-30
2      2.0          32000.0       30-35
3      2.0          60000.0       20-25
4      2.0          48000.0         35+
5      3.0              NaN         35+
6      NaN          35000.0       20-25
7      3.0          68000.0       25-30
8      4.0          91000.0       30-35
9      2.0          52000.0       30-35

After Dropping Age_Group.1:
   CustomerID Age_Group  Rating(1-5)      Hotel FoodPreference    Bill  \
0           1     20-25          4.0       Ibis            Veg  1500.0
1           2     25-30          5.0  LemonTree        Non-Veg  2100.0
```

```
2           3     30-35          3.0     RedFox            Veg  1350.0
3           4     20-25          4.0  LemonTree            Veg     NaN
4           5       35+          NaN       Ibis        Non-Veg   950.0
5           6       35+          5.0       Ibis            Veg  1800.0
6           7     20-25          4.0     RedFox        Non-Veg  1100.0
7           8     25-30          2.0  LemonTree            Veg  2400.0
8           9     30-35          5.0       Ibis        Non-Veg  3200.0
9          10     30-35          4.0     RedFox        Non-Veg  1750.0

   NoOfPax  EstimatedSalary
0      2.0          42000.0
1      3.0          56000.0
2      2.0          32000.0
3      2.0          60000.0
4      2.0          48000.0
5      3.0              NaN
6      NaN          35000.0
7      3.0          68000.0
8      4.0          91000.0
9      2.0          52000.0

Missing CustomerID:
Empty DataFrame
Columns: [CustomerID, Age_Group, Rating(1-5), Hotel, FoodPreference, Bill, NoOfPax, EstimatedSalary]
Index: []

Missing Bill:
   CustomerID Age_Group  Rating(1-5)      Hotel FoodPreference  Bill  NoOfPax  \
```

Edit with WPS Office

```
3           4     20-25          4.0  LemonTree            Veg   NaN      2.0

   EstimatedSalary
3          60000.0

Missing EstimatedSalary:
   CustomerID Age_Group  Rating(1-5) Hotel FoodPreference    Bill  NoOfPax  \
5           6       35+          5.0  Ibis            Veg  1800.0      3.0

   EstimatedSalary
5              NaN

Missing Values Count:
CustomerID         0
Age_Group          0
Rating(1-5)        1
Hotel              0
FoodPreference     0
Bill               1
NoOfPax            1
EstimatedSalary    1
dtype: int64

Unique Age Groups:
['20-25' '25-30' '30-35' '35+']
```

```
Unique Food Preferences:
['Veg' 'Non-Veg']

Food Preference after Replacement:
0        Veg
1    Non-Veg
2        Veg
3        Veg
4    Non-Veg
5        Veg
6    Non-Veg
7        Veg
8    Non-Veg
9    Non-Veg
Name: FoodPreference, dtype: object

Final Cleaned Dataset:
   CustomerID Age_Group  Rating(1-5)      Hotel FoodPreference    Bill  \
0           1     20-25          4.0       Ibis            Veg  1500.0
1           2     25-30          5.0  LemonTree        Non-Veg  2100.0
2           3     30-35          3.0    RedFox             Veg  1350.0
3           4     20-25          4.0  LemonTree            Veg  1794.0
4           5       35+          4.0       Ibis        Non-Veg   950.0
5           6       35+          5.0       Ibis            Veg  1800.0
6           7     20-25          4.0    RedFox        Non-Veg  1100.0
```

Edit with WPS Office

```
7           8     25-30          2.0  LemonTree           Veg  2400.0
8           9     30-35          5.0       Ibis       Non-Veg  3200.0
9          10     30-35          4.0     RedFox       Non-Veg  1750.0

   NoOfPax  EstimatedSalary
0      2.0          42000.0
1      3.0          56000.0
2      2.0          32000.0
3      2.0          60000.0
4      2.0          48000.0
5      3.0          53778.0
6      2.0          35000.0
7      3.0          68000.0
8      4.0          91000.0
9      2.0          52000.0
```

```
/tmp/ipykernel_2070/3845947477.py:98: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.


  df["FoodPreference"].replace(
/tmp/ipykernel_2070/3845947477.py:108: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.


  df["EstimatedSalary"].fillna(
/tmp/ipykernel_2070/3845947477.py:113: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.


  df["NoOfPax"].fillna(
/tmp/ipykernel_2070/3845947477.py:118: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.


  df["Rating(1-5)"].fillna(
/tmp/ipykernel_2070/3845947477.py:123: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.


  df["Bill"].fillna(
```

Result:

Thus, the missing, duplicate, and inappropriate data in the dataset were successfully identified and handled using Python Pandas. The missing numerical values were replaced using suitable mean and median values, and the final cleaned dataset was obtained successfully.

Edit with WPS Office

**Exp No: 14**

**Experiment to Understand Feature Scaling**

**Aim**

To understand the importance of feature scaling and apply **Standard Scaling and Min-Max Scaling** to the given dataset.

**Program Description**

The program loads the dataset and handles missing values using mean imputation. Categorical data is converted into numerical form using One-Hot Encoding. The numerical features are then scaled using **StandardScaler** and **MinMaxScaler**.

Program:

```
import numpy as np

import pandas as pd

df=pd.DataFrame({
'Country':['France','Spain','Germany','Spain','Germany','France','Spain',np.nan,'France','France'],

  'Age':[44,27,30,38,40,35,np.nan,48,50,37],

  'Salary':[72000,48000,54000,61000,np.nan,58000,52000,79000,83000,67000],

  'Purchased':['No','Yes','No','No','Yes','Yes','No','Yes','No','Yes']

})

print(df)

print(df.head())

df.Country.fillna(df.Country.mode()[0],inplace=True)

features=df.iloc[:,:-1].values

label=df.iloc[:,-1].values

from sklearn.impute import SimpleImputer

age=SimpleImputer(strategy="mean",missing_values=np.nan)

Salary=SimpleImputer(strategy="mean",missing_values=np.nan)

age.fit(features[:,[1]])

Salary.fit(features[:,[2]])
```

```
features[:,[1]]=age.transform(features[:,[1]])
features[:,[2]]=Salary.transform(features[:,[2]])
print(features)
from sklearn.preprocessing import OneHotEncoder
oh = OneHotEncoder(sparse_output=False)
Country=oh.fit_transform(features[:,[0]])
print(Country)
final_set=np.concatenate((Country,features[:,[1,2]]),axis=1)
print(final_set)
from sklearn.preprocessing import StandardScaler
sc=StandardScaler()
sc.fit(final_set)
feat_standard_scaler=sc.transform(final_set)
print(feat_standard_scaler)
from sklearn.preprocessing import MinMaxScaler
mms=MinMaxScaler(feature_range=(0,1))
mms.fit(final_set)
feat_minmax_scaler=mms.transform(final_set)
print(feat_minmax_scaler)
```

Output:

| | Country | Age | Salary | Purchased |
|---|---|---|---|---|
| 0 | France | 44.0 | 72000.0 | No |
| 1 | Spain | 27.0 | 48000.0 | Yes |
| 2 | Germany | 30.0 | 54000.0 | No |
| 3 | Spain | 38.0 | 61000.0 | No |
| 4 | Germany | 40.0 | NaN | Yes |
| 5 | France | 35.0 | 58000.0 | Yes |
| 6 | Spain | NaN | 52000.0 | No |

7 NaN 48.0 79000.0 Yes

8 France 50.0 83000.0 No

9 France 37.0 67000.0 Yes

Country Age Salary Purchased

0 France 44.0 72000.0 No

1 Spain 27.0 48000.0 Yes

2 Germany 30.0 54000.0 No

3 Spain 38.0 61000.0 No

4 Germany 40.0 NaN Yes

[['France' 44.0 72000.0]

['Spain' 27.0 48000.0]

['Germany' 30.0 54000.0]

['Spain' 38.0 61000.0]

['Germany' 40.0 63777.77777777778]

['France' 35.0 58000.0]

['Spain' 38.77777777777778 52000.0]

['France' 48.0 79000.0]

['France' 50.0 83000.0]

['France' 37.0 67000.0]]

[[1. 0. 0.]

[0. 0. 1.]

[0. 1. 0.]

[0. 0. 1.]

[0. 1. 0.]

[1. 0. 0.]

[0. 0. 1.]

[1. 0. 0.]

[1. 0. 0.]

```
 [1. 0. 0.]]
[[1.0 0.0 0.0 44.0 72000.0]
 [0.0 0.0 1.0 27.0 48000.0]
 [0.0 1.0 0.0 30.0 54000.0]
 [0.0 0.0 1.0 38.0 61000.0]
 [0.0 1.0 0.0 40.0 63777.77777777778]
 [1.0 0.0 0.0 35.0 58000.0]
 [0.0 0.0 1.0 38.77777777777778 52000.0]
 [1.0 0.0 0.0 48.0 79000.0]
 [1.0 0.0 0.0 50.0 83000.0]
 [1.0 0.0 0.0 37.0 67000.0]]
[[ 1.00000000e+00 -5.00000000e-01 -6.54653671e-01  7.58874362e-01
   7.49473254e-01]
 [-1.00000000e+00 -5.00000000e-01  1.52752523e+00 -1.71150388e+00
  -1.43817841e+00]
 [-1.00000000e+00  2.00000000e+00 -6.54653671e-01 -1.27555478e+00
  -8.91265492e-01]
 [-1.00000000e+00 -5.00000000e-01  1.52752523e+00 -1.13023841e-01
  -2.53200424e-01]
 [-1.00000000e+00  2.00000000e+00 -6.54653671e-01  1.77608893e-01
   6.63219199e-16]
 [ 1.00000000e+00 -5.00000000e-01 -6.54653671e-01 -5.48972942e-01
  -5.26656882e-01]
 [-1.00000000e+00 -5.00000000e-01  1.52752523e+00  0.00000000e+00
  -1.07356980e+00]
 [ 1.00000000e+00 -5.00000000e-01 -6.54653671e-01  1.34013983e+00
   1.38753832e+00]
 [ 1.00000000e+00 -5.00000000e-01 -6.54653671e-01  1.63077256e+00
```

1.75214693e+00]

[ 1.00000000e+00 -5.00000000e-01 -6.54653671e-01 -2.58340208e-01

2.93712492e-01]]

[[2. 0. 0. 1.47826087 1.37142857]

[0. 0. 2. 0. 0. ]

[0. 2. 0. 0.26086957 0.34285714]

[0. 0. 2. 0.95652174 0.74285714]

[0. 2. 0. 1.13043478 0.9015873 ]

[2. 0. 0. 0.69565217 0.57142857]

[0. 0. 2. 1.02415459 0.22857143]

[2. 0. 0. 1.82608696 1.77142857]

[2. 0. 0. 2. 2. ]

[2. 0. 0. 0.86956522 1.08571429]]

/tmp/ipykernel_608/3439832340.py:15: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.

The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

df.Country.fillna(df.Country.mode()[0],inplace=True)

Result:

Thus, the given dataset was successfully preprocessed and the features were scaled using StandardScaler and MinMaxScaler.

**Exp No: 15**

**Experiment to Understand Data Preprocessing in Data Science**

**Aim**

To understand the importance of data preprocessing and perform data cleaning, missing value handling, categorical encoding and label encoding on the given dataset.

**Program Description**

The program loads the dataset and handles missing values in the **Country, Age and Salary** columns. Categorical values in Country are converted into dummy variables, and the Purchased column is converted into numerical values using label encoding.

**Program**

```
import numpy as np

import pandas as pd

df=pd.DataFrame({
'Country':['France','Spain','Germany','Spain','Germany','France','Spain',np.nan,'France','Fra
nce'],

  'Age':[44,27,30,38,40,35,np.nan,48,50,37],

  'Salary':[72000,48000,54000,61000,np.nan,58000,52000,79000,83000,67000],

  'Purchased':['No','Yes','No','No','Yes','Yes','No','Yes','No','Yes']

})

print(df)

print(df.info())

print(df.Country.mode())

print(df.Country.mode()[0])

print(type(df.Country.mode()))

df.Country=df.Country.fillna(df.Country.mode()[0])

df.Age=df.Age.fillna(df.Age.median())

df.Salary=df.Salary.fillna(round(df.Salary.mean()))

print(df)

print(pd.get_dummies(df.Country))
```

```
updated_dataset=pd.concat([pd.get_dummies(df.Country),df.iloc[:,[1,2,3]]],axis=1)
print(updated_dataset)
print(df.info())
updated_dataset.Purchased=updated_dataset.Purchased.replace(['No','Yes'],[0,1])
print(updated_dataset)
```

Output:

```
    Country   Age   Salary Purchased
0   France  44.0  72000.0        No
1    Spain  27.0  48000.0       Yes
2  Germany  30.0  54000.0        No
3    Spain  38.0  61000.0        No
4  Germany  40.0      NaN       Yes
5   France  35.0  58000.0       Yes
6    Spain   NaN  52000.0        No
7      NaN  48.0  79000.0       Yes
8   France  50.0  83000.0        No
9   France  37.0  67000.0       Yes
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   Country    9 non-null      object
 1   Age        9 non-null      float64
 2   Salary     9 non-null      float64
 3   Purchased  10 non-null     object
dtypes: float64(2), object(2)
memory usage: 452.0+ bytes
None
0    France
Name: Country, dtype: object
France
<class 'pandas.core.series.Series'>
   Country   Age   Salary Purchased
0   France  44.0  72000.0        No
1    Spain  27.0  48000.0       Yes
2  Germany  30.0  54000.0        No
3    Spain  38.0  61000.0        No
4  Germany  40.0  63778.0       Yes
5   France  35.0  58000.0       Yes
```

```
    6    Spain  38.0  52000.0        No
    7   France  48.0  79000.0       Yes
...  8   France  50.0  83000.0        No
    9   France  37.0  67000.0       Yes
       France  Germany  Spain
    0    True    False  False
    1   False    False   True
    2   False     True  False
    3   False    False   True
    4   False     True  False
    5    True    False  False
    6   False    False   True
    7    True    False  False
    8    True    False  False
    9    True    False  False
       France  Germany  Spain   Age   Salary Purchased
    0    True    False  False  44.0  72000.0        No
    1   False    False   True  27.0  48000.0       Yes
    2   False     True  False  30.0  54000.0        No
    3   False    False   True  38.0  61000.0        No
    4   False     True  False  40.0  63778.0       Yes
    5    True    False  False  35.0  58000.0       Yes
    6   False    False   True  38.0  52000.0        No
    7    True    False  False  48.0  79000.0       Yes
    8    True    False  False  50.0  83000.0        No
    9    True    False  False  37.0  67000.0       Yes
    <class 'pandas.core.frame.DataFrame'>
    RangeIndex: 10 entries, 0 to 9
    Data columns (total 4 columns):
     #   Column     Non-Null Count  Dtype
    ---  ------     --------------  -----
     0   Country    10 non-null     object
     1   Age        10 non-null     float64
     2   Salary     10 non-null     float64
```

```
 2   Salary     10 non-null     float64
 3   Purchased  10 non-null     object
dtypes: float64(2), object(2)
memory usage: 452.0+ bytes
None
   France  Germany  Spain   Age   Salary  Purchased
0    True    False  False  44.0  72000.0          0
1   False    False   True  27.0  48000.0          1
2   False     True  False  30.0  54000.0          0
3   False    False   True  38.0  61000.0          0
4   False     True  False  40.0  63778.0          1
5    True    False  False  35.0  58000.0          1
6   False    False   True  38.0  52000.0          0
7    True    False  False  48.0  79000.0          1
8    True    False  False  50.0  83000.0          0
9    True    False  False  37.0  67000.0          1
/tmp/ipykernel_608/997867076.py:35: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a future version. To retain the old behavior, explicitly call `result.inf
  updated_dataset.Purchased=updated_dataset.Purchased.replace(['No','Yes'],[0,1])
```

Result:

Thus, the given dataset was successfully preprocessed by handling missing values and converting categorical data into numerical form.

# EXPERIMENT TO UNDERSTAND EDA QUANTITATIVE AND QUALITATIVE ANALYSIS

PROGRAM:

```
import warnings
warnings.filterwarnings("ignore")

import seaborn as sns
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

%matplotlib inline

tips = sns.load_dataset('tips')

tips.head()

sns.displot(tips.total_bill, kde=True)
plt.show()

sns.displot(tips.total_bill, kde=False)
plt.show()

sns.jointplot(x=tips.tip, y=tips.total_bill)
plt.show()
```

```
sns.jointplot(x=tips.tip, y=tips.total_bill, kind="reg")
plt.show()

sns.jointplot(x=tips.tip, y=tips.total_bill, kind="hex")
plt.show()

sns.pairplot(tips)
plt.show()

tips.time.value_counts()

sns.pairplot(tips, hue='time')
plt.show()

sns.pairplot(tips, hue='day')
plt.show()

sns.heatmap(tips.corr(numeric_only=True), annot=True)
plt.show()

sns.boxplot(tips.total_bill)
plt.show()

sns.boxplot(tips.tip)
plt.show()
```

```
sns.countplot(tips.day)
plt.show()

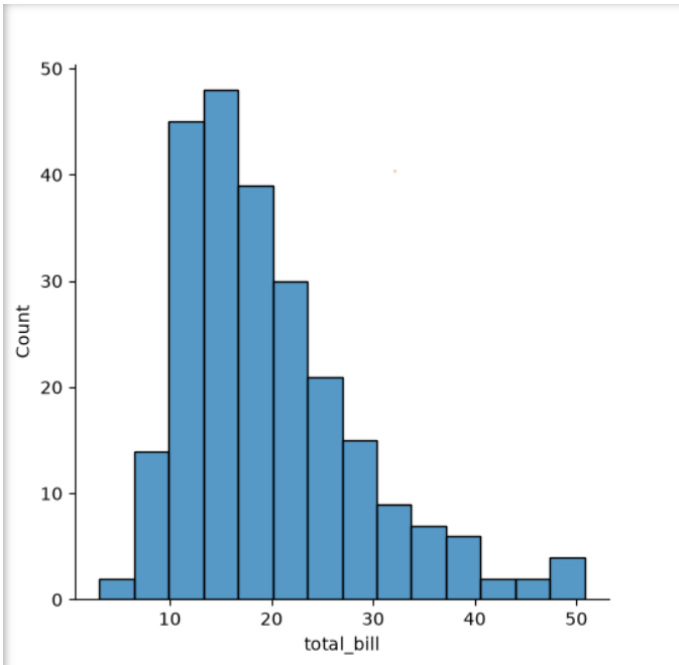
sns.countplot(tips.sex)
plt.show()

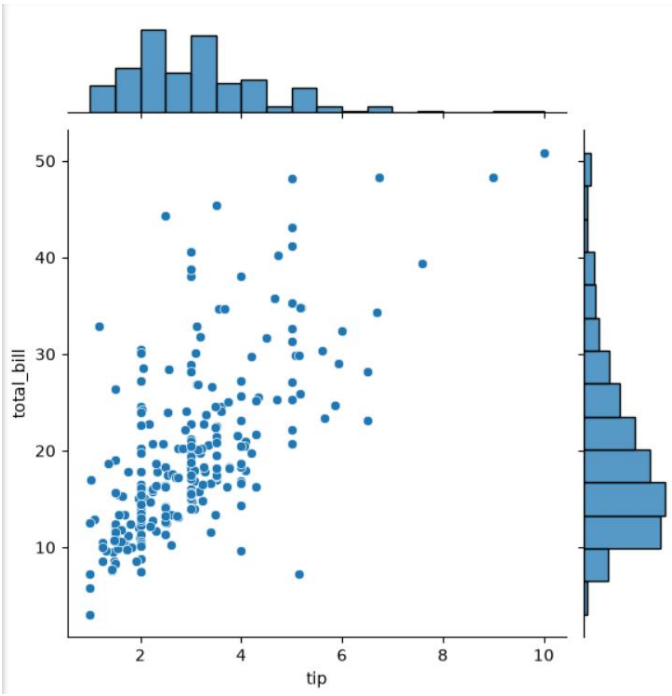
tips.sex.value_counts().plot(kind='pie')
plt.show()

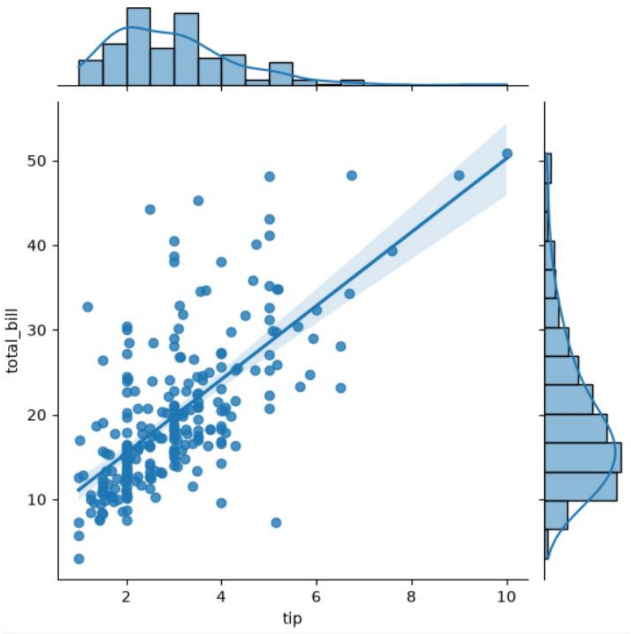
tips.sex.value_counts().plot(kind='bar')
plt.show()

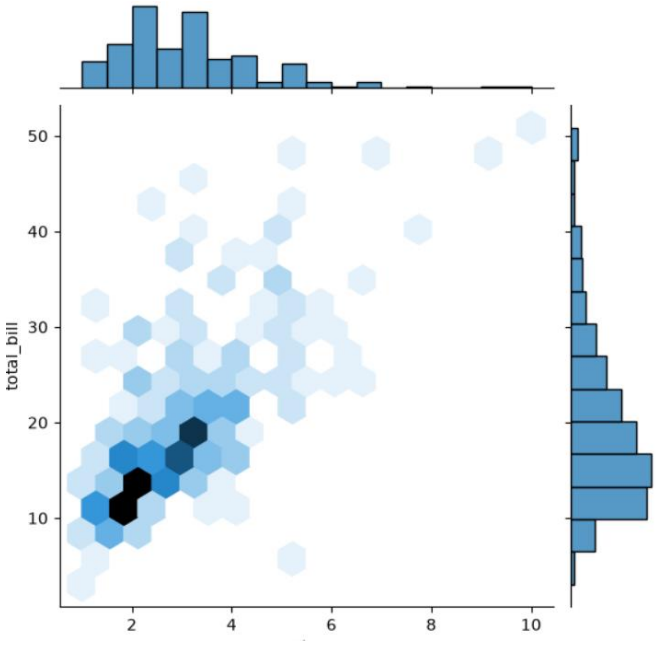
sns.countplot(tips[tips.time == 'Dinner']['day'])
plt.show()
```

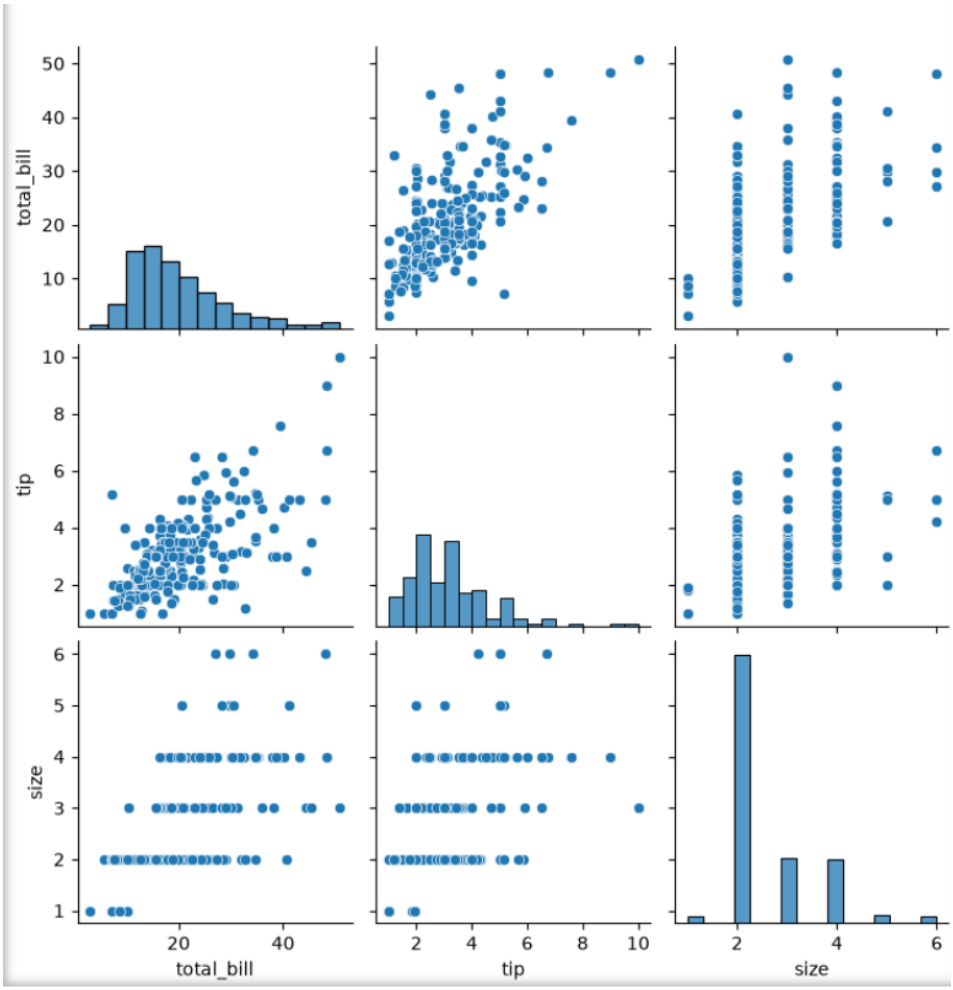
Output:

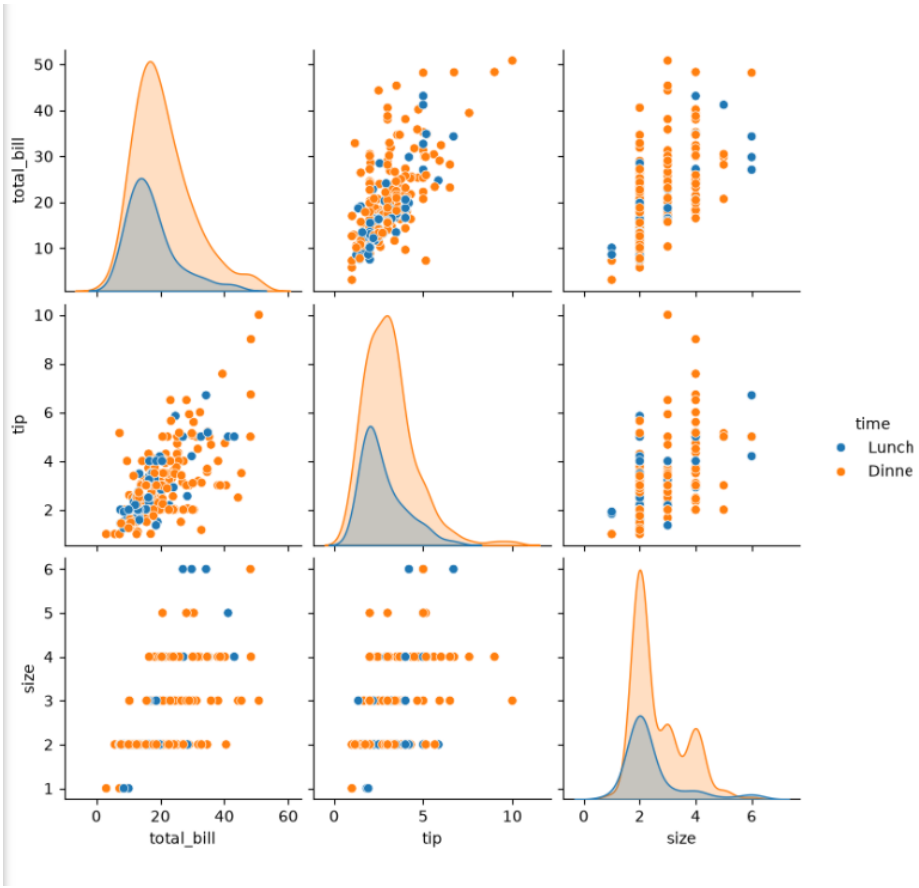


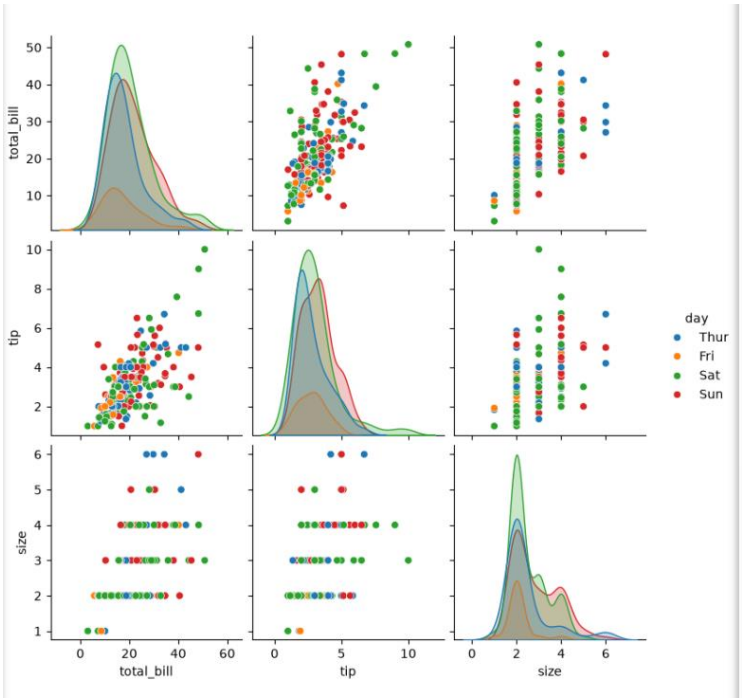

total_bill
tip
50
40
30
20
10
2
4
6
8
10

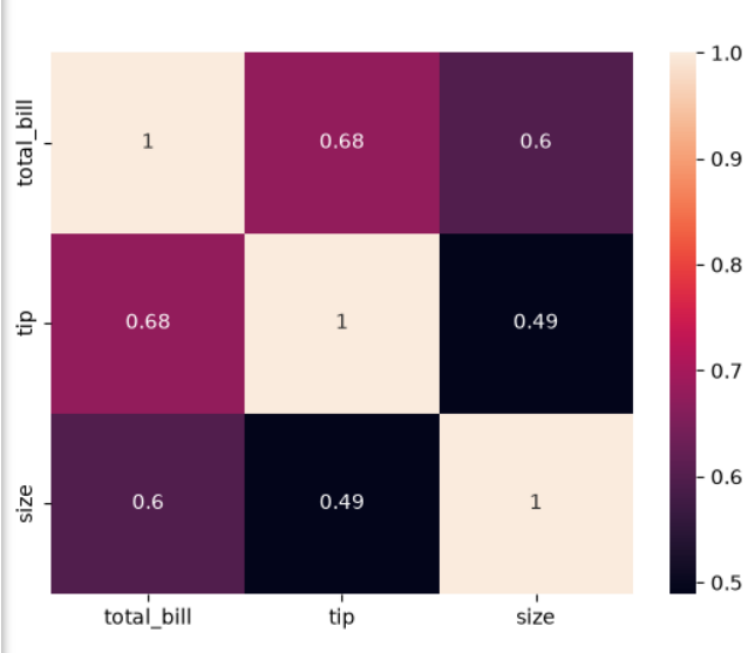

total_bill
tip
50
40
30
20
10
2
4
6
8
10

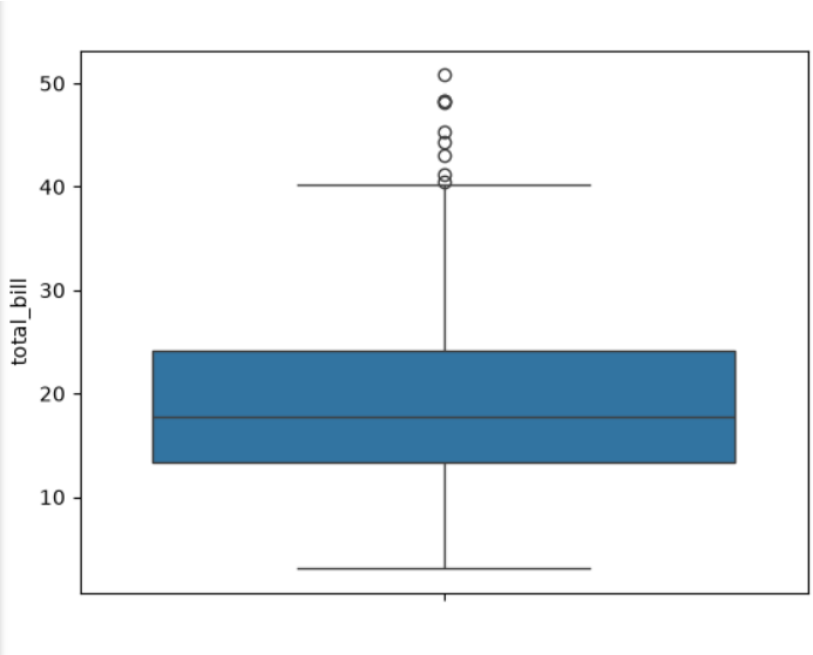

total_bill
50
40
30
20
10
2
4
6
8
10

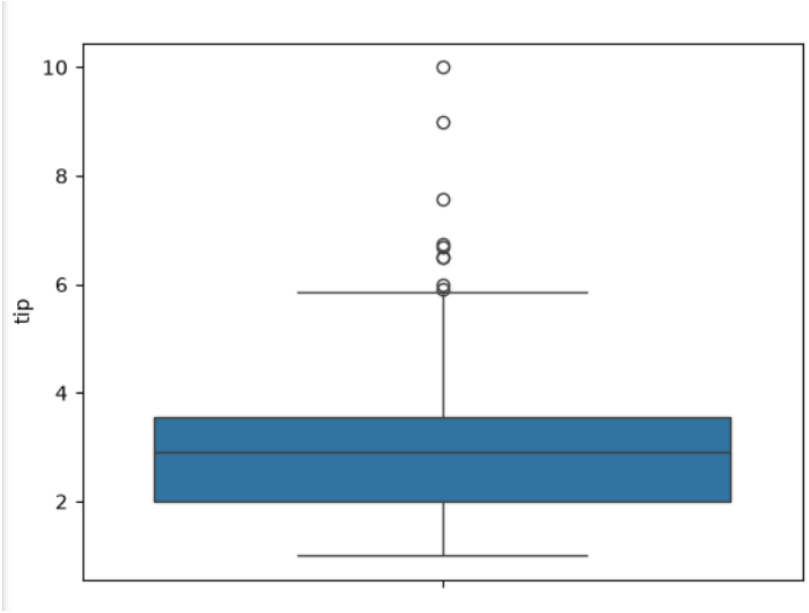

total_bill
tip
size
50
40
30
20
10
10
8
6
4
2
6
5
4
3
2
1
20
40
2
4
6
8
10
2
4
6

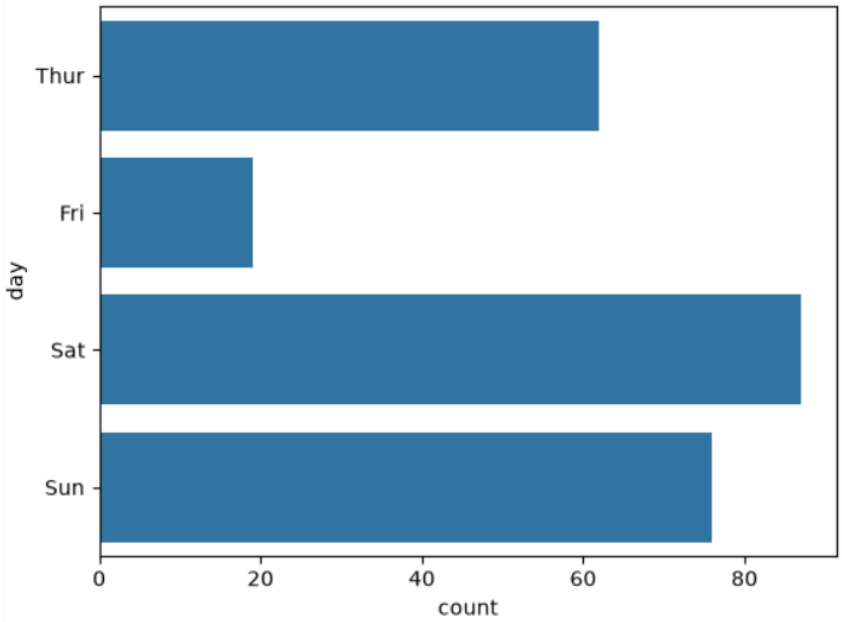

total_bill
tip
size
total_bill
tip
size
time
Lunch
Dinne

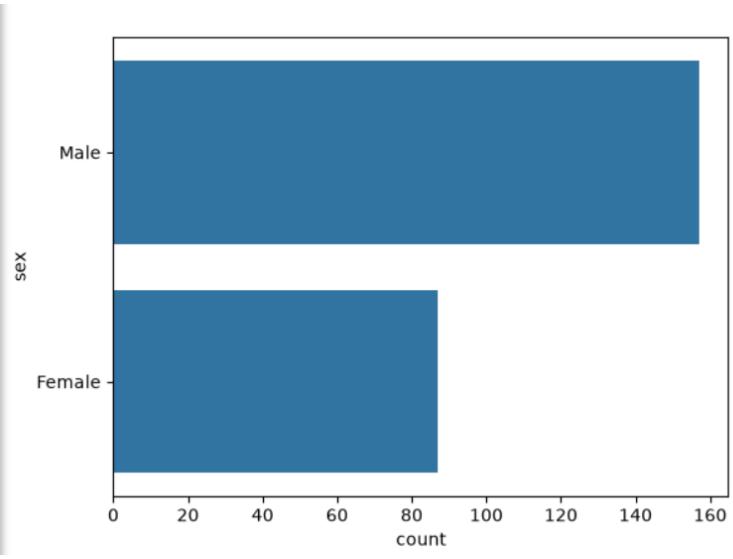

total_bill
tip
size
total_bill
tip
size
day
Thur
Fri
Sat
Sun

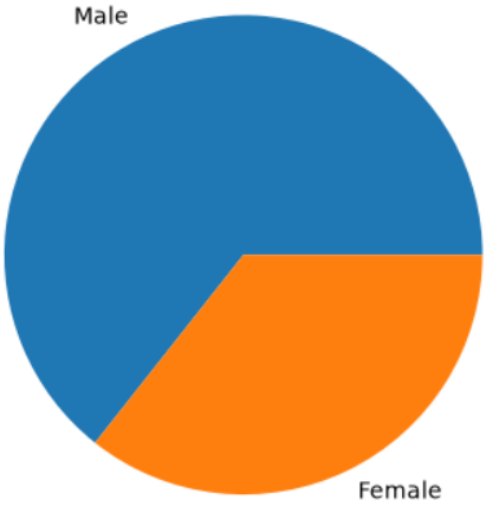

total_bill
tip
size
1
0.68
0.6
0.68
1
0.49
0.6
0.49
1
total_bill
tip
size
1.0
0.9
0.8
0.7
0.6
0.5

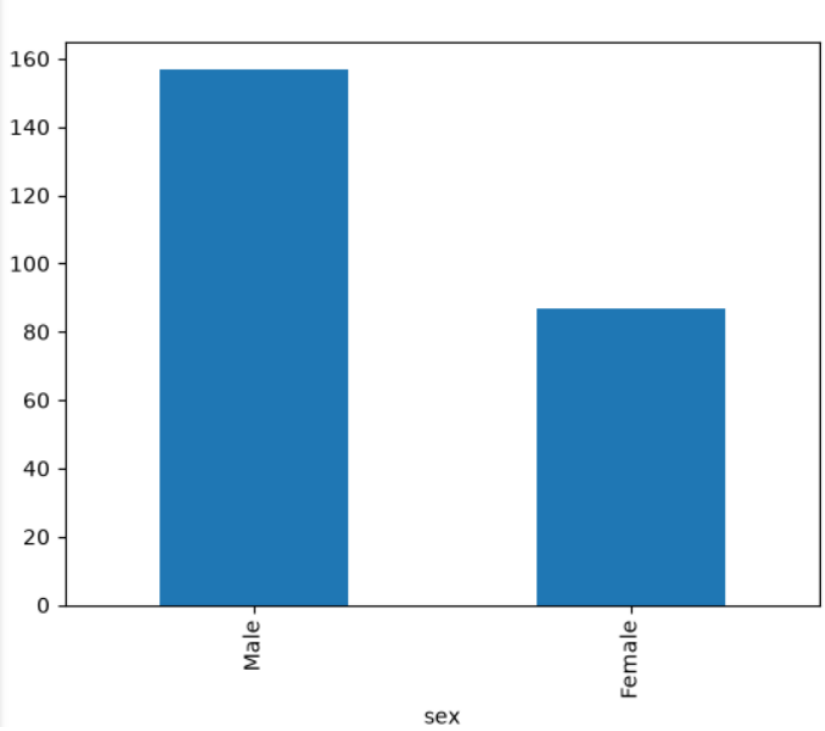

total_bill
50
40
30
20
10

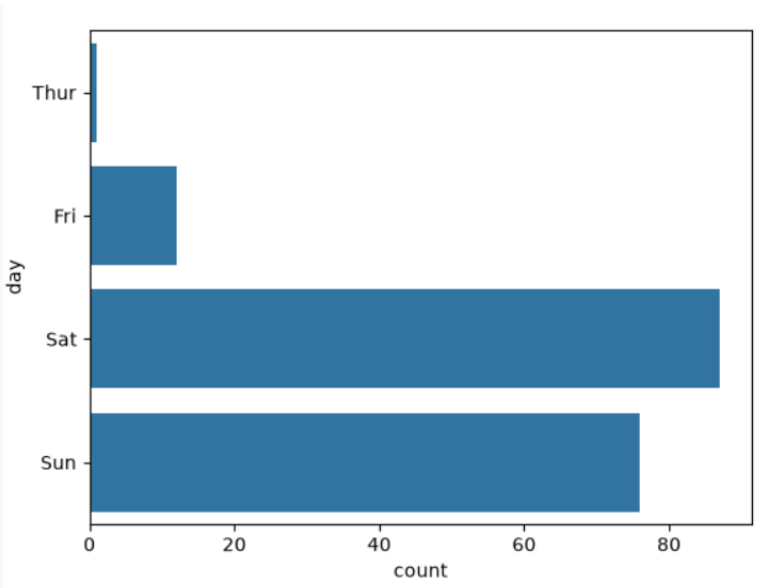

10
8
6
4
2
tip


Thur
Fri
Sat
Sun
day
0
20
40
60
80
count


Male
Female
sex
0
20
40
60
80
100
120
140
160
count


Male
Female


160
140
120
100
80
60
40
20
0
Male
Female
sex


Thur
Fri
Sat
Sun
day
0
20
40
60
80
count

# EXPERIMENT TO UNDERSTAND LINEAR REGRESSION FOR A GIVEN DATA SET

PROGRAM:

```
import numpy as np
import pandas as pd

df=pd.read_csv('Salary_data.csv')
print(df)

df.info()

df.dropna(inplace=True)

df.info()

print(df.describe())

features=df.iloc[:,[0]].values
label=df.iloc[:,[1]].values

from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(features,label,test_size=0.2,random_state=0)

from sklearn.linear_model import LinearRegression
model=LinearRegression()
```

```
model.fit(x_train,y_train)


print(model.score(x_train,y_train))
print(model.score(x_test,y_test))
print(model.coef_)
print(model.intercept_)


import pickle
pickle.dump(model,open('SalaryPred.model','wb'))


model=pickle.load(open('SalaryPred.model','rb'))


yr_of_exp=float(input("Enter Years of Experience: "))
yr_of_exp_NP=np.array([[yr_of_exp]])
Salary=model.predict(yr_of_exp_NP)


print("Estimated Salary for {} years of experience is {}: " .format(yr_of_exp,Salary))
```

OUTPUT:

```
  YearsExperience  Salary
0               1.1   39200
1               1.3   42100
2               1.5   44800
3               2.0   47200
4               2.2   50100
5               2.9   54800
6               3.0   56600
7               3.2   59100
8               3.7   62400
9               3.9   64600
10              4.0   67100
11              4.1   68900
12              4.5   72400
```

```
13              4.7   74800
14              5.0   78100
15              5.2   80600
16              5.3   83500
17              5.9   87200
18              6.0   89900
19              6.8   95100
20              7.1   97800
21              7.9  103500
22              8.2  107200
23              8.7  112400
24              9.0  116000
25              9.5  120500
26              9.6  123800
27             10.0  127500
28             10.3  131200
29             10.5  135000
<class 'pandas.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 2 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   YearsExperience  30 non-null     float64
 1   Salary           30 non-null     int64
dtypes: float64(1), int64(1)
memory usage: 612.0 bytes
<class 'pandas.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 2 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   YearsExperience  30 non-null     float64
 1   Salary           30 non-null     int64
dtypes: float64(1), int64(1)
memory usage: 612.0 bytes
       YearsExperience         Salary
count         30.00000      30.000000
mean           5.57000   83113.333333
std            2.92211   28988.734237
min            1.10000   39200.000000
25%            3.32500   59925.000000
50%            5.10000   79350.000000
75%            8.12500  106275.000000
max           10.50000  135000.000000
0.9965480200876051
0.9986651018711702
[[9917.57201802]]
[27760.44923723]
```

```
Enter Years of Experience:  5

Estimated Salary for 5.0 years of experience is [[77348.30932733]]:
```

# EXPERIMENT TO UNDERSTAND KNN ALGORITHMN FOR A GIVEN DATA SET

PROGRAM:

```
import numpy as np
import pandas as pd

df = pd.read_csv('Iris_modified.csv')
df
df.info()
df.variety.value_counts()
df.head()
features = df.iloc[:, :-1].values
label = df.iloc[:, 4].values
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

xtrain, xtest, ytrain, ytest = train_test_split(
    features, label, test_size=0.2, random_state=42
)
print(model_KNN.score(xtrain, ytrain))
print(model_KNN.score(xtest, ytest))
from sklearn.metrics import confusion_matrix

confusion_matrix(label, model_KNN.predict(features))
from sklearn.metrics import classification_report
```

print(classification_report(label, model_KNN.predict(features)))

OUTPUT:

| | sepal.length | sepal.width | petal.length | petal.width | variety |
|---|---|---|---|---|---|
| **0** | 5.2 | 3.5 | 1.5 | 0.3 | Setosa |
| **1** | 4.7 | 3.2 | 1.5 | 0.3 | Setosa |
| **2** | 4.6 | 3.2 | 1.2 | 0.4 | Setosa |
| **3** | 4.7 | 3.2 | 1.5 | 0.1 | Setosa |
| **4** | 5.0 | 3.4 | 1.5 | 0.3 | Setosa |
| **...** | ... | ... | ... | ... | ... |
| **145** | 6.7 | 3.0 | 5.0 | 2.3 | Virginica |
| **146** | 6.5 | 2.6 | 4.9 | 1.8 | Virginica |
| **147** | 6.4 | 2.8 | 5.4 | 2.0 | Virginica |
| **148** | 6.1 | 3.4 | 5.2 | 2.2 | Virginica |
| **149** | 6.0 | 3.1 | 5.3 | 1.8 | Virginica |

150 rows × 5 columns

```
<class 'pandas.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   sepal.length  150 non-null    float64
 1   sepal.width   150 non-null    float64
 2   petal.length  150 non-null    float64
 3   petal.width   150 non-null    float64
 4   variety       150 non-null    str
dtypes: float64(4), str(1)
memory usage: 6.0 KB
```

---

```
variety
Setosa        50
Versicolor    50
Virginica     50
Name: count, dtype: int64
```

| | sepal.length | sepal.width | petal.length | petal.width | variety |
|---|---|---|---|---|---|
| **0** | 5.2 | 3.5 | 1.5 | 0.3 | Setosa |
| **1** | 4.7 | 3.2 | 1.5 | 0.3 | Setosa |
| **2** | 4.6 | 3.2 | 1.2 | 0.4 | Setosa |
| **3** | 4.7 | 3.2 | 1.5 | 0.1 | Setosa |
| **4** | 5.0 | 3.4 | 1.5 | 0.3 | Setosa |

```
0.9416666666666667
0.9666666666666667
```

```
array([[50,  0,  0],
       [ 0, 48,  2],
       [ 0,  6, 44]])
```

```
  precision    recall  f1-score   support

      Setosa       1.00      1.00      1.00        50
  Versicolor       0.89      0.96      0.92        50
   Virginica       0.96      0.88      0.92        50

    accuracy                           0.95       150
   macro avg       0.95      0.95      0.95       150
weighted avg       0.95      0.95      0.95       150
```

# EXPERIMENT TO UNDERSTAND LOGISTIC RGRESSION FOR A DATASET

Program:

```
 import numpy as np
import pandas as pd

df = pd.read_csv('Social_Network_Ads_modified.csv')
df
df.head()
features = df.iloc[:, [2, 3]].values
label = df.iloc[:, 4].values

Features
Label
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
x_train, x_test, y_train, y_test = train_test_split(
    features, label, test_size=0.2, random_state=37
model = LogisticRegression()

model.fit(x_train, y_train)
) train_score = model.score(x_train, y_train)
test_score = model.score(x_test, y_test)

print("Train Accuracy:", train_score)
```

print("Test Accuracy:", test_score)

from sklearn.metrics import classification_report

print(classification_report(label, model.predict(features)))

OUTPUT:

| | User ID | Gender | Age | EstimatedSalary | Purchased | |
|---|---|---|---|---|---|---|
| **0** | 17000001 | Male | 41 | 74570 | 1 | |
| **1** | 17000002 | Female | 28 | 19015 | 0 | |
| **2** | 17000003 | Male | 25 | 79813 | 0 | |
| **3** | 17000004 | Male | 53 | 45712 | 0 | |
| **4** | 17000005 | Male | 55 | 118497 | 1 | |
| **...** | ... | ... | ... | ... | ... | ... |
| **395** | 17000396 | Female | 22 | 57986 | 1 | |
| **396** | 17000397 | Female | 42 | 103595 | 1 | |
| **397** | 17000398 | Female | 19 | 119358 | 1 | |
| **398** | 17000399 | Female | 27 | 82929 | 1 | |
| **399** | 17000400 | Male | 47 | 91506 | 1 | |

400 rows × 5 columns

| | User ID | Gender | Age | EstimatedSalary | Purchased | |
|---|---|---|---|---|---|---|
| **0** | 17000001 | Male | 41 | 74570 | 1 | |
| **1** | 17000002 | Female | 28 | 19015 | 0 | |
| **2** | 17000003 | Male | 25 | 79813 | 0 | |
| **3** | 17000004 | Male | 53 | 45712 | 0 | |
| **4** | 17000005 | Male | 55 | 118497 | 1 | |

---

```
array([[    41,  74570],
       [    28,  19015],
       [    25,  79813],
       [    53,  45712],
       [    55, 118497],
       [    57, 130816],
       [    37, 146778],
       [    52,  80292],
       [    42,  41833],
       [    52,  38309],
       [    42,  24970],
       [    46,  90474],
```

```
[    35,  64540],
[    35,  36017],
[    19,  93880],
[    52, 123570],
[    33,  26702],
[    58, 149456],
[    53,  18404],
[    50,  30763],
[    21, 116142],
[    50,  98642],
[    31,  68612],
[    38,  41574],
[    37,  41524],
[    25,  28916],
[    24,  22809],
[    20, 139288],
[    34,  56765],
[    50,  63106],
[    29,  71932],
[    39,  75091],
[    39,  39949],
[    47,  55744],
[    55, 126319],
[    55, 110193],
[    25,  54187],
[    44, 142454],
[    44,  32489],
[    51,  79877],
[    38,  67922],
[    47, 112074],
[    50,  55131],
[    45, 133765],
[    50,  91963],
[    22, 124884],
[    36, 143917],
[    21,  42285],
[    52,  55626],
[    34,  37198],
[    45,  33254],
[    47,  62238],
[    46,  19252],
[    23,  57764],
[    52,  95709],
[    58,  71682],
[    54,  77494],
[    41,  99317],
[    46,  83215],
[    48,  96587],
[    52,  24949],
[    50,  73393],
```

```
[    38,  43934],
[    49,  93745],
[    40, 112456],
[    50,  71413],
[    20, 115239],
[    35,  60318],
[    42,  88390],
[    59,  61088],
[    48, 121196],
[    20,  73479],
[    57,  37963],
[    41,  73123],
[    49, 143911],
[    39, 120468],
[    40,  91477],
[    19,  32388],
[    44,  23569],
[    59,  87836],
[    19, 122573],
[    43,  63002],
[    34,  42071],
[    57,  76243],
[    50, 143827],
[    26, 126215],
[    60, 119570],
[    56,  87724],
[    46, 133359],
[    59,  93697],
[    43,  46732],
[    52, 139465],
[    42,  75108],
[    41, 129195],
[    30, 121081],
[    24,  54668],
[    53,  41061],
[    37,  97252],
[    18,  57749],
[    25, 148560],
[    33, 122680],
[    31,  69745],
[    29, 126464],
[    40, 109865],
[    32, 130428],
[    45,  41624],
[    51,  41321],
[    19,  40574],
[    49, 136831],
[    40, 113257],
[    39,  82291],
[    42, 145600],
```

```
[    39,  68428],
[    39, 149374],
[    59, 149127],
[    23, 129317],
[    32, 145449],
[    60,  46699],
[    54,  91686],
[    50,  85661],
[    25,  62247],
[    22, 123113],
[    56,  22621],
[    21,  36030],
[    23, 100838],
[    49, 102717],
[    47,  56638],
[    52,  85067],
[    57, 136033],
[    33, 123977],
[    30, 106869],
[    59, 123186],
[    47,  52674],
[    36,  64271],
[    34,  91688],
[    36, 129076],
[    45,  19062],
[    43,  68448],
[    54, 113462],
[    43,  65915],
[    40,  22632],
[    26, 122592],
[    29, 127779],
[    18, 126099],
[    18,  25561],
[    51, 100152],
[    49, 143036],
[    42, 142049],
[    57,  44734],
[    18,  44069],
[    33,  30910],
[    56,  91479],
[    22,  99264],
[    39,  41275],
[    46,  90274],
[    20, 118776],
[    29,  66447],
[    43, 144394],
[    33,  93703],
[    54,  94090],
[    39,  35260],
[    46, 130581],
```

```
[    31,  21812],
[    45, 134148],
[    22, 134421],
[    47,  19605],
[    22,  90303],
[    29,  94125],
[    33, 142040],
[    43,  22324],
[    43,  94323],
[    38,  27111],
[    56,  34389],
[    53, 140177],
[    50,  59427],
[    47, 140883],
[    54, 149189],
[    40, 143808],
[    27, 130221],
[    22,  31234],
[    53,  71441],
[    51,  18846],
[    48,  28954],
[    27,  97239],
[    36,  64514],
[    49, 106236],
[    18, 115579],
[    22,  60101],
[    21,  82323],
[    33, 109922],
[    41, 145534],
[    33,  85584],
[    19, 122537],
[    45,  96477],
[    49,  60533],
[    44,  19828],
[    37,  19917],
[    41,  38618],
[    29,  50254],
[    52, 123292],
[    50,  65362],
[    50,  38128],
[    60, 141781],
[    54,  83545],
[    29, 100180],
[    20, 105958],
[    18,  57809],
[    50, 134748],
[    57,  23287],
[    27,  26512],
[    60, 150414],
[    46, 126872],
```

```
[    30, 111070],
[    29,  97767],
[    48, 115829],
[    19,  60078],
[    52, 124288],
[    40, 101524],
[    34,  30219],
[    43,  18235],
[    25,  60929],
[    46, 145309],
[    43,  85444],
[    27, 119834],
[    43, 102555],
[    51,  91698],
[    58, 142450],
[    24,  61925],
[    21, 105388],
[    28,  26007],
[    46,  76596],
[    53, 139101],
[    42, 106861],
[    38, 104740],
[    53,  33524],
[    27, 149149],
[    54, 143501],
[    26,  38611],
[    41,  32060],
[    52,  61303],
[    52,  57711],
[    53,  91340],
[    35, 120974],
[    56,  85728],
[    49,  29246],
[    41, 145796],
[    40, 110263],
[    49,  29302],
[    54,  85506],
[    29,  59157],
[    30,  72917],
[    40, 114646],
[    42, 101122],
[    52,  42519],
[    58,  98910],
[    47, 138430],
[    34,  89324],
[    37,  23408],
[    42,  77379],
[    39,  43053],
[    30, 149994],
[    36, 116723],
```

```
[    53,  69356],
[    29,  28789],
[    58, 115720],
[    36,  84656],
[    29,  62028],
[    26,  82484],
[    24,  50154],
[    45,  20336],
[    31, 121677],
[    48,  85779],
[    36,  37029],
[    33,  77201],
[    22,  50265],
[    52, 120916],
[    29,  64197],
[    42, 143733],
[    38,  89907],
[    53, 146876],
[    40,  82435],
[    33,  58631],
[    56, 147038],
[    59,  38103],
[    56,  82772],
[    31, 144108],
[    48,  32382],
[    22, 102291],
[    52, 119195],
[    40,  38609],
[    46,  93683],
[    60, 142714],
[    28, 148330],
[    35,  89315],
[    29,  95834],
[    26, 144205],
[    27,  32153],
[    34,  94325],
[    55,  73318],
[    24, 149886],
[    30, 133033],
[    57,  94077],
[    59, 108474],
[    26,  91319],
[    44, 143885],
[    19, 127777],
[    22,  35368],
[    46, 109751],
[    54,  45306],
[    55,  55219],
[    36,  78279],
[    25,  71028],
```

```
[    18, 122263],
[    39,  18126],
[    34,  70329],
[    24,  45632],
[    42, 127945],
[    21,  73176],
[    53,  52560],
[    23,  94589],
[    48,  40116],
[    36, 150717],
[    56, 142085],
[    44, 135718],
[    27,  37256],
[    43, 118158],
[    36, 137719],
[    56,  86244],
[    20,  74264],
[    30,  75545],
[    45,  57224],
[    37, 115945],
[    45,  31224],
[    25,  58738],
[    58,  81731],
[    56,  26070],
[    18, 144161],
[    20,  26666],
[    30,  75761],
[    45,  54368],
[    42,  59914],
[    50, 145738],
[    55,  85526],
[    23,  37956],
[    49,  64240],
[    38, 139008],
[    33, 112270],
[    38,  22757],
[    28,  37097],
[    54, 121892],
[    53,  88480],
[    52,  64163],
[    36,  37816],
[    37,  70853],
[    35, 114731],
[    58,  80823],
[    31,  19759],
[    32, 110374],
[    48,  42442],
[    18, 111501],
[    20,  20079],
[    33,  77676],
```

```
       [    40, 108397],
       [    28,  90847],
       [    29, 111607],
       [    27,  82077],
       [    49,  56258],
       [    33, 137771],
       [    25,  56000],
       [    55,  44737],
       [    29,  29485],
       [    41,  56565],
       [    45, 137614],
       [    25,  83155],
       [    45,  70487],
       [    53, 147948],
       [    43,  30342],
       [    25,  58806],
       [    45,  66770],
       [    45,  96623],
       [    54, 121177],
       [    58, 149026],
       [    53,  93629],
       [    44, 138993],
       [    34, 144944],
       [    26, 123947],
       [    50,  44399],
       [    37, 146177],
       [    30, 127575],
       [    45, 128904],
       [    46,  48678],
       [    30,  94218],
       [    52,  81230],
       [    23,  92055],
       [    35,  90258],
       [    22,  57986],
       [    42, 103595],
       [    19, 119358],
       [    27,  82929],
       [    47,  91506]]) array([1, 0, 0, 0, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 0, 1, 1,
1, 1, 1,
       1, 0, 1, 0, 0, 1, 0, 1, 1, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0,
       1, 1, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 1, 0, 0, 1, 1, 0, 1, 0, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 0, 1, 1, 0, 1, 1, 0, 1, 0, 0, 1, 1, 1, 1, 1, 0, 1, 0,
       1, 1, 1, 0, 1, 0, 1, 1, 1, 1, 0, 1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 1,
       1, 0, 0, 0, 1, 0, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1,
       1, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 0, 1, 0, 1, 1, 1, 0, 0, 1, 1, 1,
       1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 1, 0, 1, 0, 0, 1, 1, 0, 0, 1, 1,
       1, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 0, 0, 0, 0, 1, 1, 0, 0,
       1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 0, 1, 1, 0, 0, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1, 0, 0,
```

```
       1, 1, 0, 1, 0, 0, 1, 0, 1, 1, 0, 1, 0, 1, 1, 1, 1, 0, 0, 1, 1, 1,
       1, 0, 1, 0, 1, 0, 1, 1, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0,
       0, 1, 1, 0, 1, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 1, 0, 1, 1, 1, 0, 0,
       1, 0, 1, 1, 0, 1, 0, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 1, 0,
       0, 0, 0, 1, 0, 1, 1, 0, 0, 0, 1, 1, 1, 0, 1, 1, 0, 1, 0, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1,
       1, 1, 1, 1])
```

---

```
x_train, x_test, y_train, y_test = train_test_split(
    features, label, test_size=0.2, random_state=37
)
Train Accuracy: 0.7375
Test Accuracy: 0.675

precision    recall  f1-score   support

           0       0.63      0.29      0.40       125
           1       0.74      0.92      0.82       275

    accuracy                           0.72       400
   macro avg       0.69      0.61      0.61       400
weighted avg       0.71      0.72      0.69       400
```

# EXPERIMENT TO UNDERSTAND K-MEANS CLUSTRING ALGORITHMN FOR A GIVEN DATASET

PROGRAM:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

%matplotlib inline

df = pd.read_csv('Mall_Customers_modified.csv')
df.info()
df.head()
sns.pairplot(df)
features = df.iloc[:, [3, 4]].values
Features
from sklearn.cluster import KMeans

model = KMeans(n_clusters=5)
model.fit(features)
Final = df.iloc[:, [3, 4]].copy()
Final['label'] = model.predict(features)
Final.head()
sns.set_style("whitegrid")
```

```
sns.FacetGrid(Final, hue="label", height=8) \
.map(plt.scatter, "Annual Income (k$)", "Spending Score (1-100)") \
.add_legend()

plt.show()
features_el = df.iloc[:, [2, 3, 4]].values

from sklearn.cluster import KMeans

wcss = []

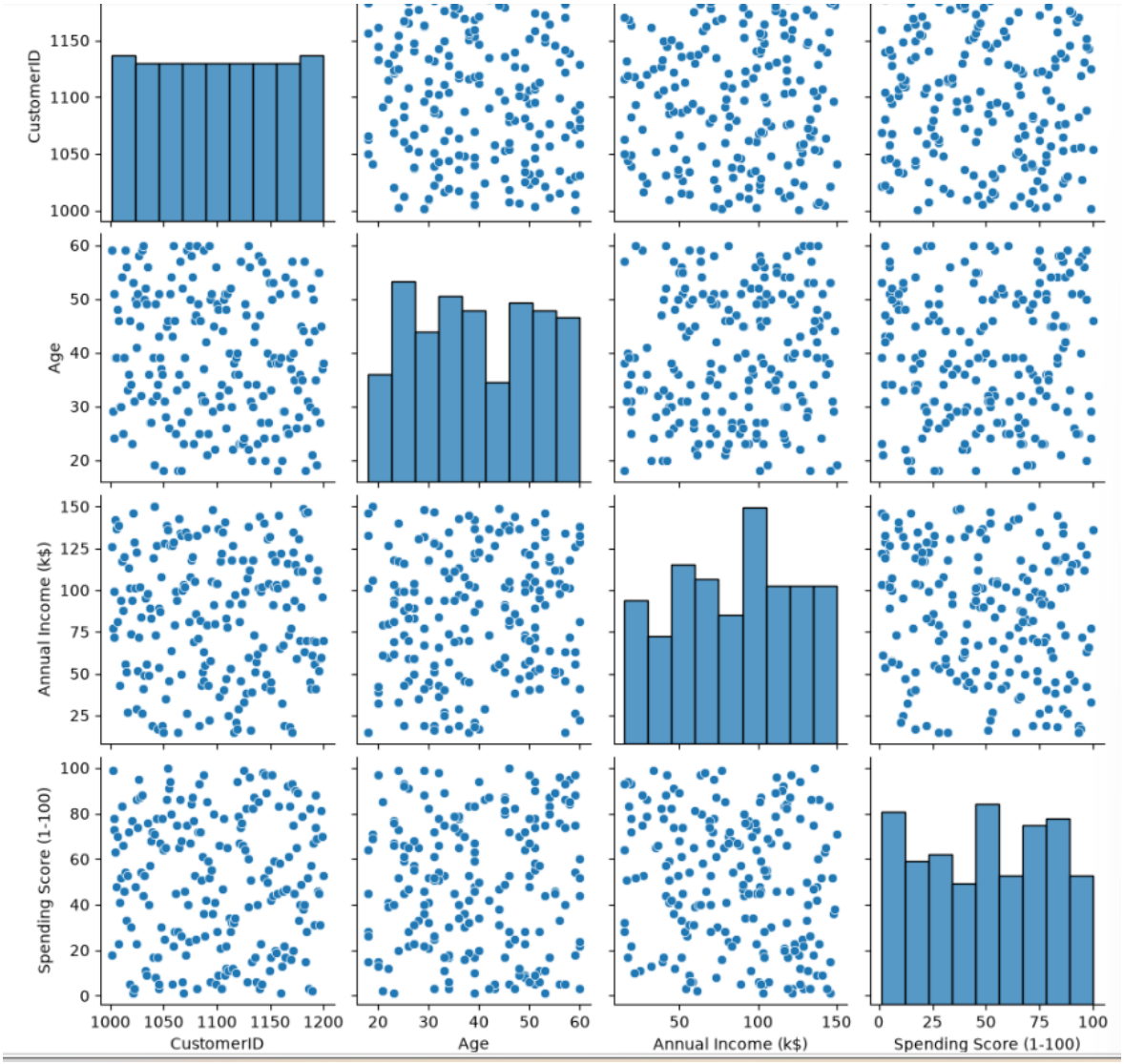
for i in range(1, 10):
    model = KMeans(n_clusters=i)
    model.fit(features_el)
    wcss.append(model.inertia_)
plt.plot(range(1, 10), wcss)
plt.xlabel("Number of Clusters")
plt.ylabel("WCSS")
plt.show()
```

OUTPUT:

```
<class 'pandas.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   CustomerID              200 non-null    int64
 1   Gender                  200 non-null    str
 2   Age                     200 non-null    int64
 3   Annual Income (k$)      200 non-null    int64
 4   Spending Score (1-100)  200 non-null    int64
dtypes: int64(4), str(1)
```

```
memory usage: 7.9 KB
```

| | CustomerID | Gender | Age | Annual Income (k$) | Spending Score (1-100) |
|---|---|---|---|---|---|
| **0** | 1001 | Female | 59 | 126 | 18 |
| **1** | 1002 | Male | 29 | 77 | 99 |
| **2** | 1003 | Female | 24 | 99 | 73 |
| **3** | 1004 | Male | 51 | 72 | 78 |
| **4** | 1005 | Female | 39 | 142 | 63 |



```
array([[126,  18],
       [ 77,  99],
       [ 99,  73],
       [ 72,  78],
       [142,  63],
       [137,  48],
       [ 81,  70],
       [139,  23],
       [ 43,  41],
       [ 94,  52],
       [117,  83],
       [ 88,  66],
       [120,  46],
       [ 56,  54],
       [ 51,  33],
       [ 27,  53],
       [113,  72],
       [101,   5],
       [ 74,  30],
       [ 94,  77],
       [146,   1],
       [129,   3],
       [101,  48],
       [ 29,  86],
       [123,  23],
       [ 52,  74],
       [ 72,  95],
       [102,  87],
       [ 84,  54],
       [ 26,  88],
       [ 41,  44],
       [ 49,  53],
       [ 95,  11],
       [ 56,   9],
       [ 98,  76],
       [ 49,  40],
       [ 89,  57],
       [ 83,  68],
       [ 19,  72],
       [119,  17],
       [150,  71],
       [ 73,   8],
       [127,  78],
       [ 17,  17],
       [ 54,   3],
       [ 89,   5],
       [ 87,  67],
       [108,  30],
       [ 19,  78],
```

```
[ 15,  64],
[128,  25],
[ 35,  65],
[139,  86],
[136, 100],
[ 46,  91],
[127,  94],
[ 64,  80],
[127,   5],
[129,  22],
[ 79,  28],
[ 99,  45],
[ 94,  75],
[ 15,  28],
[143,  65],
[134,  86],
[133,  26],
[ 50,  68],
[100,   6],
[103,   1],
[102,  46],
[135,  18],
[ 26,  75],
[132,  40],
[ 81,  24],
[108,  67],
[118,  84],
[120,  87],
[ 69,  77],
[105,  53],
[ 86,  25],
[133,   3],
[ 71,  72],
[ 19,  93],
[ 82,  72],
[ 43,  51],
[ 51,  61],
[ 46,  36],
[ 63,  97],
[ 55,  26],
[137,  42],
[ 79,  85],
[ 43,  59],
[ 22,  52],
[ 58,   6],
[105,  55],
[148,  36],
[ 91,  80],
[ 80,  41],
[104,   5],
```

```
[104,  28],
[137,   9],
[ 36,  66],
[117,  21],
[122,   3],
[135,  47],
[ 40,  78],
[141,   9],
[ 83,  22],
[ 47,  12],
[ 78,  65],
[ 25,  11],
[ 92,  34],
[ 53,  28],
[118,  32],
[127,  12],
[ 15,  32],
[ 97,  34],
[ 21,  10],
[ 17,  94],
[ 29,  79],
[ 81,  67],
[ 46,  74],
[ 92,  76],
[118,  66],
[ 33,  99],
[ 99,  64],
[ 38,  82],
[107,   6],
[138,  60],
[ 60,  39],
[112,  96],
[ 16,  51],
[ 39,  15],
[118,  82],
[ 73,  87],
[ 51,  17],
[ 57,   6],
[ 62,  40],
[101,  85],
[144,   3],
[ 99,  23],
[102,   5],
[ 66,  98],
[140,  52],
[ 42,  97],
[ 50,  89],
[131,  50],
[104,  55],
[132,  11],
```

```
[ 40,  17],
[ 45,  42],
[121,  97],
[ 91,  44],
[102,  59],
[117,  20],
[103,  19],
[101,  45],
[145,  83],
[ 65,  58],
[122,   1],
[ 32,  13],
[ 67,  46],
[ 19,  22],
[104,  17],
[ 90,  47],
[116,  92],
[ 73,  61],
[ 18,  83],
[ 77,  16],
[ 15,  93],
[123,  20],
[135,  75],
[ 94,  45],
[115,  90],
[ 59,  65],
[111,  89],
[131,  33],
[ 70,  40],
[ 90,  49],
[111,  79],
[149,  38],
[ 63,  40],
[146,  15],
[ 70,  28],
[147,  52],
[119,   3],
[ 45,  82],
[ 41,  57],
[ 61,   2],
[ 70,  67],
[ 56,  31],
[ 69,  74],
[ 41,  88],
[106,  69],
[112,  46],
[ 52,  45],
[ 59,  31],
[ 60,  81],
```

```
       [ 96,  70],
       [ 70,  53]])
```

| | Annual Income (k$) | Spending Score (1-100) | label | |
|---|---|---|---|---|
| **0** | 126 | 18 | 1 | |
| **1** | 77 | 99 | 2 | |
| **2** | 99 | 73 | 2 | |
| **3** | 72 | 78 | 2 | |
| **4** | 142 | 63 | 3 | |

---

